

Chapter – 3

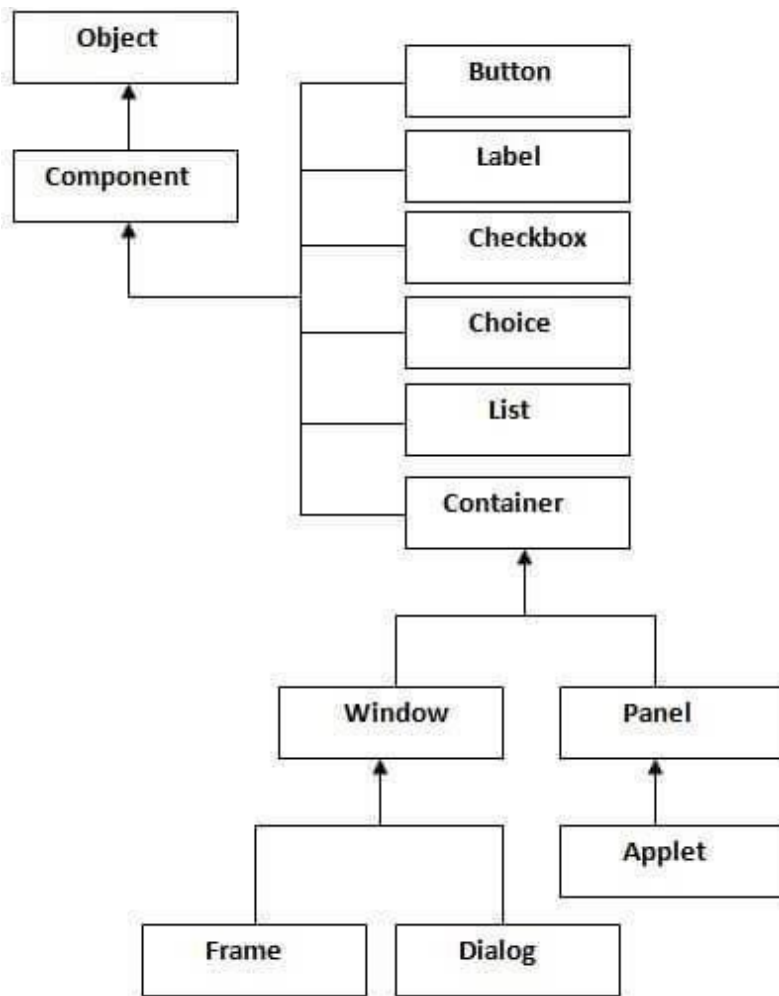
Building Components using Swing and JavaFX

AWT and Swing both are used to create GUI interface in Java.

Java AWT

Java AWT(Abstract Window Toolkit) is an API to develop GUI or window based applications in java. AWT components are platform – dependent i.e components are displayed according to the view of operating system. AWT is a heavyweight i.e its components are using the resources of OS. The java.awt package provides classes for AWT API such as TextField, TextArea, RadioButton, CheckBox, Choice, List etc.

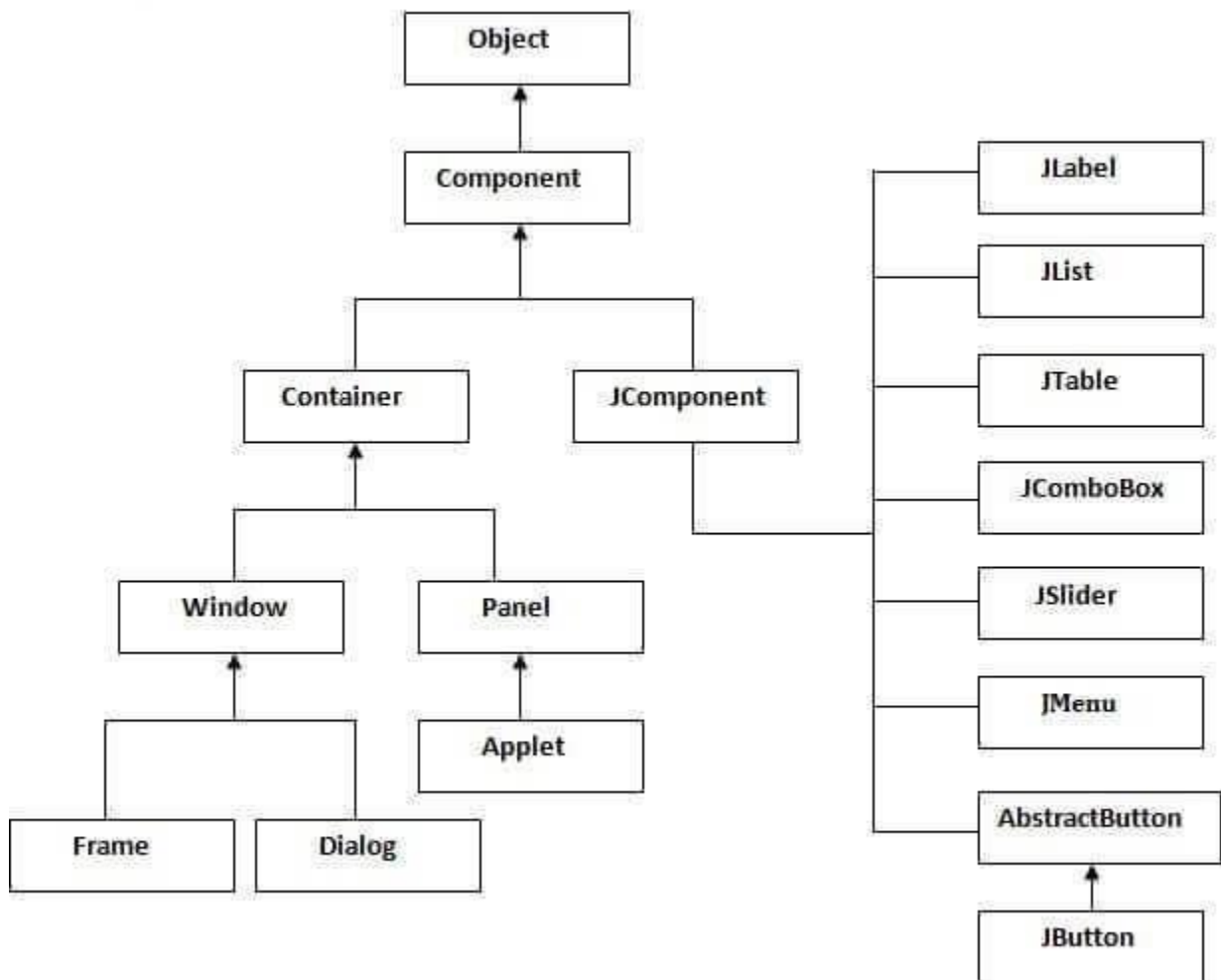
Java AWT hierarchy



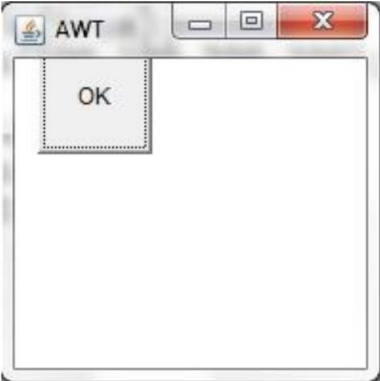
Java Swing

Java Swing is a part of java Foundation Classes (JFC) that is used to create window – based applications. It is built on the top of AWT (Abstract Windowing Toolkit) API and entirely written in java. Unlike AWT, java Swing provides platform – independent and lightweight components. The java,swing package provides classes for java swing API such as JButton, JTextField, JTextArea, JRadioButton, JCheckBox, JMenu, JColorChooser etc.

Java Swing Hierarchy



Difference between java AWT and Swing

AWT	Swing
AWT stands for Abstract Window Toolkit.	Swing is a part of Java Foundation Class (JFC).
AWT components are heavy weight.	Swing components are light weight.
AWT components are platform dependent so there look and feel changes according to OS.	Swing components are platform independent so there look and feel remains constant.
AWT components are not very good in look and feel as compared to Swing components. See the button in below image, its look is not good as button created using Swing.	Swing components are better in look and feel as compared to AWT. See the button in below image, its look is better than button created using AWT.
	

JFrame

- Whenever you create a graphical user interface with java swing functionality, you will need a container for your application. In this case of swing, this container is called a JFrame. All GUI applications requires a JFrame. In fact, some Applets even use a JFrame. Why?
- You can't build a house without a foundation. The same is true in java: without a container in which to put all other elements, you won't have a GUI application. In other words, the JFrame is required as the foundation or base container for all other graphical components.
- Java Swing applications can be run on any system that supports Java. These applications are lightweight. This means that don't take up much space or use many system resources.
- JFrame is a class in java and has its own methods and constructors. Methods are functions that impact the JFrame, such as setting the size or visibility. Constructors are run when the instance is created: One constructor can create a blank JFrame, while another can create it with a default title.
- When a new JFrame is created, you actually create an instance of the JFrame class. You can create an empty one, or one with a title. If you pass a string into the constructor, a title is created as follows

JFrame f = new JFrame();

Or overload the constructor and give it a title.

```
JFrame f1 = new JFrame("its me Nishanta");
```

Jpanel in Swing

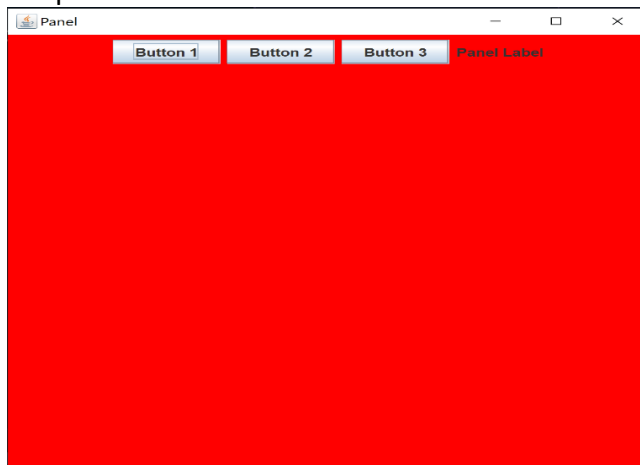
The JPanel is a simplest container class. It provides space in which an application can attach any other component. It inherits the JComponents class. It doesn't have title bar

```
JFrame f= new JFrame("Panel Example");  
JPanel panel=new JPanel();  
JLabel lbl=new JLabel("Testing label");  
panel.add(lbl);  
f.add(panel);
```

Example:

```
import java.awt.*;
import javax.swing.*;
class PanelDemo extends JFrame
{
    static JFrame f;
    static JButton b1,b2,b3;
    static JLabel l;
    public static void main(String args[])
    {
        f = new JFrame("Panel");
        l = new JLabel("Panel Label");
        b1 = new JButton("Button 1");
        b2 = new JButton("Button 2");
        b3 = new JButton("Button 3");
        JPanel p = new JPanel();
        p.add(b1);
        p.add(b2);
        p.add(b3);

        p.add(l);
        p.setBackground(Color.red);
        f.add(p);
        f.setSize(500, 500);
        f.show();
    }
}
```

Output:

Swing Components and Containers

A component is an independent visual control. Swing Framework contains a large set of components which provide rich functionalities and allow high level of customization. They all are derived from JComponent class. All these components are lightweight components. This class provides some common functionality like pluggable look and feel, support for accessibility, drag and drop, layout etc.

A container holds a group of components. It provides a space where a components can be managed and displayed. Containers are of two types

1. Top level Containers

- It inherits Component and Container of AWT.
- It cannot be contained within other container.
- Heavyweight
- Example: JFrame, JDialog, JApplet

2. Lightweight Container

- It inherits JComponent class.
- It is a general purpose container.
- It can be used to organize related components together.
- Example: JPanel

JPanel

JPanel, a part of java Swing package is a container that can store a group of components. The main task of JPanel is to organize components; various layouts can be set in JPanel which provides better organization of components and however it does not have a title bar.

Constructor of JPanel are:

- `JPanel()`: Creates a new panel with flow layout.
- `JPanel(LayoutManager l)`: Creates a new JPanel with specified layoutManager.
- `JPanel(boolean isDoubleBuffered)`: Creates a new JPanel with a specified buffering strategy.
- `JPanel(LayoutManager l, boolean isDoubleBuffered)`: Creates a new JPanel with specified layoutManager and a specified buffering strategy.

Commonly used function:

- `Add(Component c)`: adds component to a specified container.
- `SetLayout(LayoutManager l)`: Sets the layout of the container to specified layout manager.
- `UpdateUI()`: resets the UI property with a value from the current look and feel.
- `SetUI(PanelsUI ui)`: sets the look and feel object that renders this component.
- `GetUI()`: returns the look and feel object that renders this component.
- `paramString()`: returns a string representation of this JPanel.
- `GetUIClassID()`: returns the name of the look and feel class that renders this component.
- `GetAccessibleContext()`: gets the AccessibleContext associated with this JPanel.

JButton:

JButton is an implementation of a push button. It is used to trigger an action if the user clicks on it. JButton can display a text, an icon or both.

```
JButton bt = new JButton("Click Me");
```

Example

```
import java.awt.*;
import javax.swing.*;
class MyButtonFrame extends JFrame
{
    MyButtonFrame() // Use the class name for the constructor
    {
        JButton bt1 = new JButton("Yes"); // Creating yes button
        JButton bt2 = new JButton("NO"); // Creating No button
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());
        setSize(400, 400); // Setting size of JFrame
        add(bt1); // adding yes button to frame
        add(bt2); // adding No button to frame
        setVisible(true);
    }

    public static void main(String args[])
    {
        {
            new MyButtonFrame(); // Create an instance of your class
        }
    }
}
```

Output

JLabel

Label is a simple component for displaying text, images or both. It doesnot react to input events.

```
JLabel lb = new Label("This is a label");
```

JTextField

JTextField is a text component that allows editing of a single line of non – formatted text.

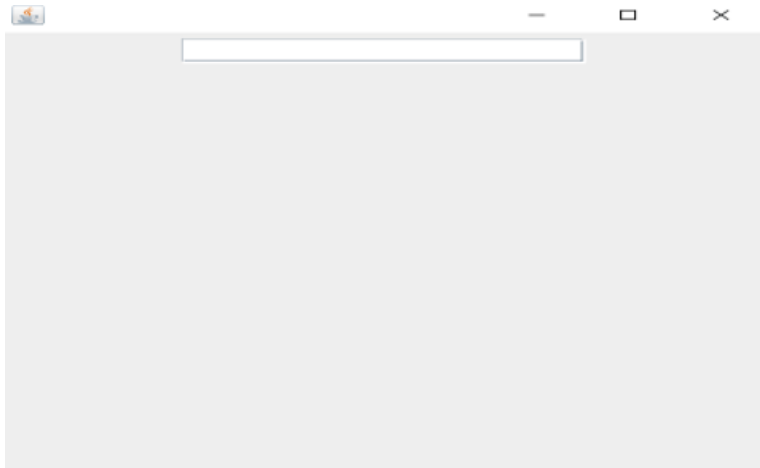
```
JTextField txt = new JTextField();
```

Example:

```
import java.awt.*;
import javax.swing.*;
class MyTextField extends JFrame
{
    public MyTextField() // Use the class name for the constructor
    {
        JTextField tf = new JTextField(20); // creating JTextField
        add(tf);
        setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(400, 400);
        setVisible(true);
    }

    public static void main(String args[])
    {
        {
            new MyTextField(); // Create an instance of your class
        };
    }
}
```

Output:



JRadioButton

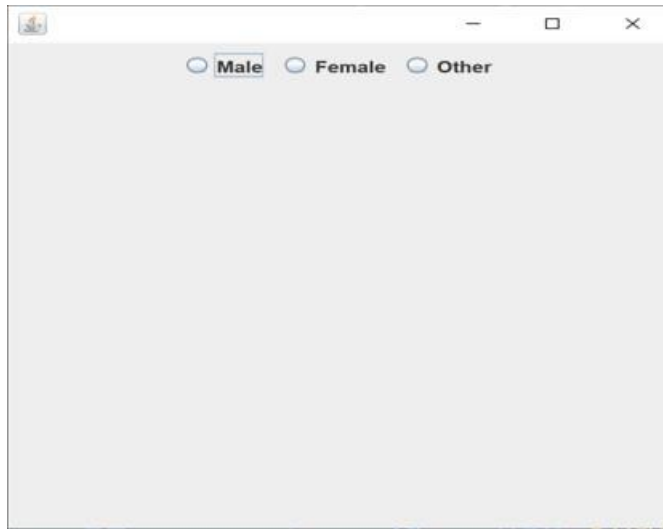
JRadioButton allows the user to select a single exclusive choice from a group of options. It is used with the ButtonGroup Component.

```
JRadioButton rb1 = new JRadioButton("Male");
JRadioButton rb2 = new JRadioButton("Female");
ButtonGroup bg = new ButtonGroup();
bg.add(rb1);
bg.add(rb2);
```

Example:

```
1  import java.awt.*;
2  import javax.swing.*;
3
4  class MyRadioButton extends JFrame
5  {
6      MyRadioButton() // Use the class name for the constructor
7      {
8          JRadioButton jb = new JRadioButton("Male"); // Creating JRadioButton
9          add(jb); //adding JRadioButton to frame
10         jb = new JRadioButton("Female"); //creating JRadioButton
11         add(jb); //adding JRadioButton to frame
12         jb = new JRadioButton("Other"); //creating JRadioButton
13         add(jb); //adding JRadioButton to frame
14         setLayout(new FlowLayout());
15         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
16         setSize(400,400);
17         setVisible(true);
18     }
19
20     public static void main(String args[])
21     {
22         new MyRadioButton(); // Create an instance of your class
23     }
24 }
```

Output:



A screenshot of a web browser window. The window has a title bar with a small icon on the left and standard minimize, maximize, and close buttons on the right. The main content area is light gray. At the top of the content area, there are three radio buttons. The first radio button is selected and is followed by the text 'Male'. The second radio button is followed by the text 'Female'. The third radio button is followed by the text 'Other'.

JComboBox

JComboBox is a component that combines a button or editable field and drop – down list. The user can select a value from the drop – down list, which appears at the user's request. If you make the combo box editable, then the combo box includes an editable field into which the user can type a value.

```
String arr[] = {"BCA", "BBA", "BBS", "BSC"};
```

```
JComboBox cb = new JComboBox(arr);
```

Or

```
JComboBox<String> cb = new JComboBox<String>();
```

```
cb.addItem("BCA");
```

```
cb.addItem("BBA");
```

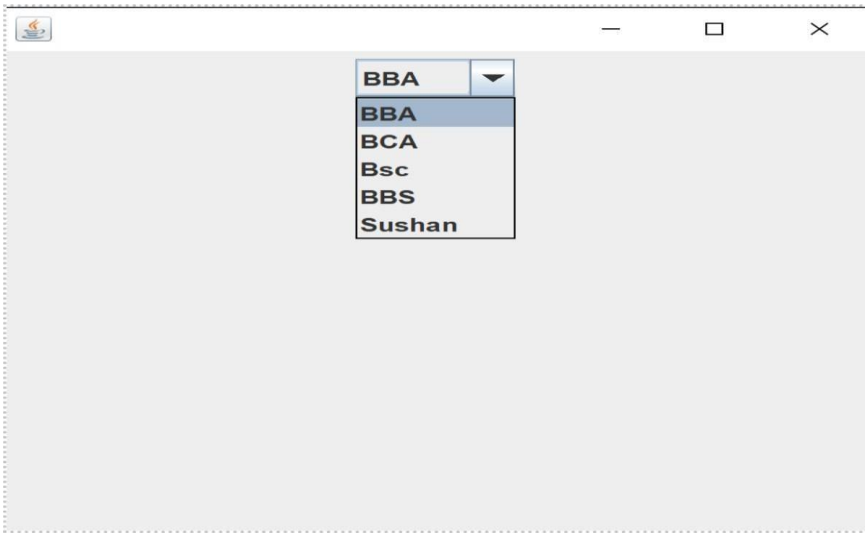
```
cb.addItem("BBS");
```

```
cb.addItem("BSC");
```

Example

```
1  ∨ import java.awt.*;
2  import javax.swing.*;
3
4  ∨ class MyComboBox extends JFrame
5  {
6      String faculty[] = {"BBA", "BCA", "Bsc", "BBS", "Sushan"};
7  ∨  MyComboBox()
8      {
9          JComboBox cb = new JComboBox<>(faculty);
10         add(cb); // adding JComboBox to frame
11         setLayout(new FlowLayout());
12         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
13         setSize(width:400,height:400);
14         setVisible(b:true);
15     }
16  ∨  Run | Debug
17  public static void main(String args[])
18  {
19      {
20          new MyComboBox(); // Create an instance of your class
21      }
22  }
```

Example



JList:

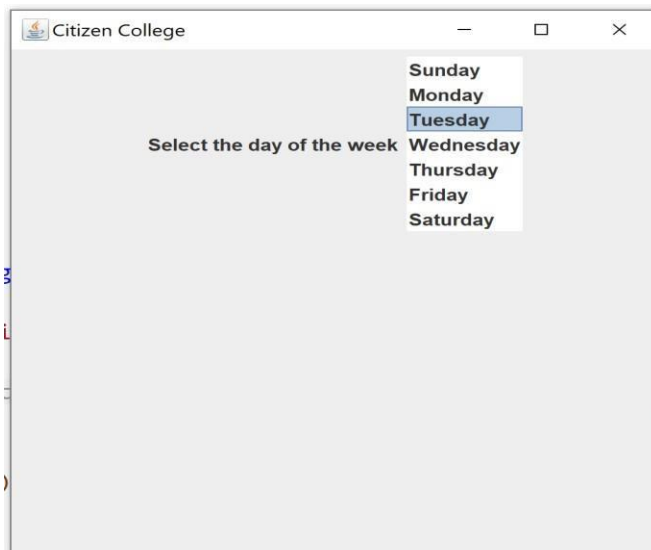
JList is a component that displays a list of objects. It allows the user to select one or more items.

```
String arr[] = {"BCA", "BBA", "BBS", "BSC"};
```

```
JList cb = new JList(arr);
```

```
1  import java.awt.*;
2  import javax.swing.*;
3
4  class MyJlist extends JFrame
5  {
6      static JFrame f;
7      static JList l;
8
9      public static void main(String args[])
10     {
11         f = new JFrame(title:"Citizen College");
12         JPanel p = new JPanel();
13         JLabel lb = new JLabel(text:"Select the day of the week");
14         String week[] = {"Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"};
15         l = new JList<>(week);
16         l.setSelectedIndex(index:2);
17         p.add(lb);
18         p.add(l);
19         f.add(p);
20         f.setSize(width:400,height:400);
21         f.setVisible(b:true);
22     }
23 }
```

Output



JTextArea

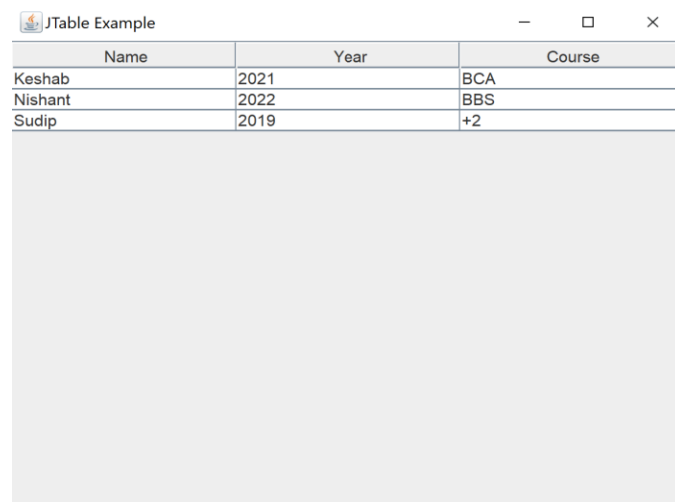
A JTextArea is a multiline text area that displays plain text. It is lightweight component for working with text. The component does not handle scrolling. For this task, we use JScrollPane component.

```
JTextArea txt = new JTextArea();
```

JTable

The JTable class is a part of java Swing Package and is generally used to display or edit two dimensional data that is having both rows and columns. It is similar to a spreadsheet. This arranges data in a tabular form.

```
1 import java.awt.*;
2 import javax.swing.*;
3
4 class Jtable1
5 {
6     JFrame f;
7     JTable j;
8     String[] columnNames = {"Name", "Year", "Course"};
9
10    Jtable1()
11    {
12        f = new JFrame();
13        f.setTitle(title:"JTable Example");
14        String data[][] = {{"Keshab", "2021", "BCA"}, {"Nishant", "2022", "BBS"}, {"Sudip", "2019", "+2"}};
15        j = new JTable(data, columnNames);
16        JScrollPane sp = new JScrollPane(j);
17        f.add(sp);
18        f.setSize(width:500, height:400);
19        f.setVisible(b:true);
20    }
21
22    Run | Debug
23    public static void main(String args[])
24    {
25        new Jtable1();
26    }
27 }
```



Name	Year	Course
Keshab	2021	BCA
Nishant	2022	BBS
Sudip	2019	+2

JMenu

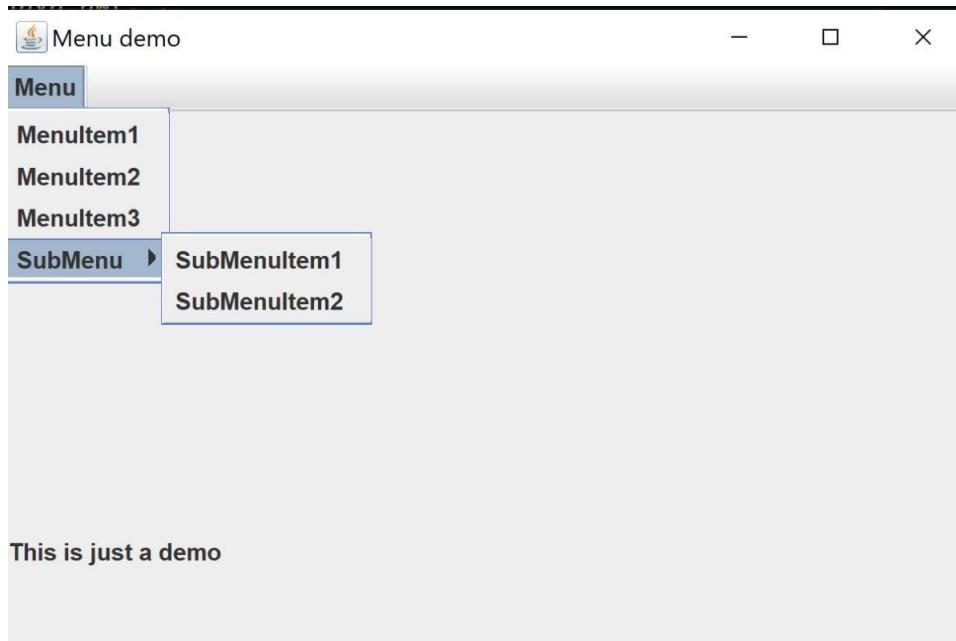
The JMenuBar class is used to display menubar on the window or frame. It may have several menus.

The object of JMenu class is a pull down menu component which is displayed from the menu bar. It inherits the JMenuItem class.

The object of JMenuItem class adds a simple labeled menu item. The item used in a menu must belong to the JMenuItem or any of its subclass.

```
JMenuBar mb = new JMenuBar();  
JMenu menu1 = new JMenu("Courses");  
JMenuItem item1 = new JMenuItem("BCA");  
JMenuItem item2 = new JMenuItem("BBA");  
menu1.add(item1);  
menu1.add(item2);  
mb.add(menu1);
```

```
1  √ import java.awt.*;
2    import javax.swing.*;
3
4  √ class MenuDemo extends JFrame
5  {
6      static JFrame f;
7      static JLabel l;
8      static JMenuBar mb;
9      static JMenu x1, x2;
10     static JMenuItem m1, m2, m3, s1, s2;
11
12     Run | Debug
13     √ public static void main(String args[]) {
14         f = new JFrame(title:"Menu demo");
15         l = new JLabel(text:" This is just a demo");
16         mb = new JMenuBar();
17         x1 = new JMenu(s:"Menu");
18         x2 = new JMenu(s:"SubMenu");
19         m1 = new JMenuItem(text:"MenuItem1");
20         m2 = new JMenuItem(text:"MenuItem2");
21         m3 = new JMenuItem(text:"MenuItem3");
22         s1 = new JMenuItem(text:"SubMenuItem1");
23         s2 = new JMenuItem(text:"SubMenuItem2");
24
25         x1.add(m1);
26         x1.add(m2);
27         x1.add(m3);
28
29         x2.add(s1);
30         x2.add(s2);
31
32         x1.add(x2);
33
34         mb.add(x1);
35
36         f.setJMenuBar(mb);
37         f.add(l);
38         f.setSize(width:500, height:500);
39         f.setVisible(b:true);
40     }
```



Dialog Boxes in swing

Dialog windows or dialogs are an indispensable part of most modern GUI applications. A dialog is defined as a conversation between two or more persons. In a computer application a dialog is a window which is used to “talk” to the application. A dialog is used to input data, modify data, change the application settings etc. Dialogs are important means of communication between a user and computer program.

In java Swing, we can create two kinds of dialogs: standard dialogs and custom dialogs. Custom dialogs are created by programmers. They are based on the `JDialog` class. Standard dialogs are predefined dialogs available in the Swing toolkit, for example the `JColorChooser` or the `JFileChooser`. These are dialogs for common programming tasks like showing text, receiving input, loading and saving files. They save programmer’s time and enhance using some standard behaviour.

Modal and Modeless Dialog

There are two basic types of dialogs: Modal and Modeless.

Modal dialogs block input to other top – level windows. Modeless dialogs allow input to other windows. An open file dialog is a good example of a modal dialog. While choosing a file to open, no other operation should be permitted.

A typical modeless dialog is a find text dialog. It is handy to have the ability to move the cursor in the text control and define, where to start the finding of the particular text.

Standard Dialog Example – Message Box

Message dialogs are simple dialogs that provide information to the user. Message dialogs are created with the `JOptionPane.showMessageDialog()` method.

```
JOptionPane.showMessageDialog(null,"This is a test message");
```

Custom Dialog Creation

```
JDialog jd = new JDialog();  
Jd.setTitle("This is a test message");  
JLabel lb = new JLabel("Do you want to exit ? ");  
Jd.add(lb);  
JButton yes = new JButton("Yes");  
Jd.add(yes);
```

```
1  import javax.swing.*;  
2  class SimpleDialog  
3  {  
4      Run | Debug  
5      public static void main(String args[])  
6      {  
7          JFrame frame = new JFrame(title:"Main Window");  
8          JOptionPane.showMessageDialog(frame, message:"Message for the dialog box goes here.",title:"Error", JOptionPane.ERROR_MESSAGE);  
9          frame.setSize(width:350,height:350);  
10         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
11         frame.setVisible(b:true);  
12     }  
}
```



Layout Management

FlowLayout

The FlowLayout arranges the components in a directional flow, either from left to right or from right to left. Normally all components are set to one row, according to the order of different components, if all components cannot be fit into one row, it will start a new row and fit the rest in.

To construct a FlowLayout, three options could be chosen:

- **FlowLayout():** Construct a new FlowLayout object with center alignment and horizontal and vertical gap to be default size of 5 pizel.

- **FlowLayout(int align):** Construct similar object with different settings on alignment.
- **FlowLayout(int align, int hgap, int vgap):** Construct similar object with different settings on alignment and gaps between components.

```

1  import javax.swing.*;
2  import java.awt.*;
3  public class FlowLayoutDemo
4  {
5      JFrame f1;
6      JButton b1,b2,b3,b4,b5,b6,b7,b8;
7      FlowLayoutDemo()
8      {
9          f1 = new JFrame();
10         f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11         f1.setSize(width:400,height:400);
12         f1.setLayout(new FlowLayout(align:0,hgap:15,vgap:15));
13
14         b1 = new JButton(text:"Button 1");
15         b2 = new JButton(text:"Button 2");
16         b3 = new JButton(text:"Button 3");
17         b4 = new JButton(text:"Button 4");
18         b5 = new JButton(text:"Button 5");
19         b6 = new JButton(text:"Button 6");
20         b7 = new JButton(text:"Button 7");
21         // b8 = new JButton("Button 8");
22
23         f1.add(b1);
24         f1.add(b2);
25         f1.add(b3);
26         f1.add(b4);
27         f1.add(b5);
28         f1.add(b6);
29         f1.add(b7);
30         //f1.add(b8);
31
32         f1.setVisible(b:true);
33     }
34     Run | Debug
35     public static void main(String args[])
36     {
37         new FlowLayoutDemo();
38     }
39 }

```

Output



GridLayout

The GridLayout manager is used to lay out the components in a rectangle grid, which has been divided into equal – sized rectangles and one component is placed in each rectangle.

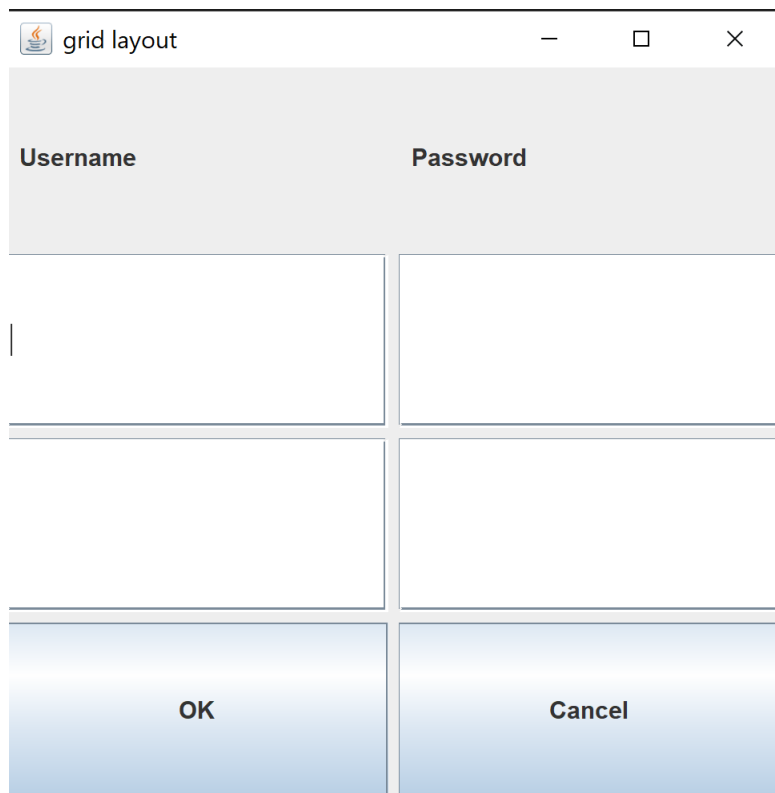
It can be constructed with following methods

- **GridLayout():** Construct a grid layout with one column per component in a single row.
- **GridLayout(int row, int col):** Construct a grid layout with specified numbers of rows and columns.
- **GridLayout(int row, int col, int hgap, int vgap):** Construct a grid layout with specified rows, columns and gaps between components.

```

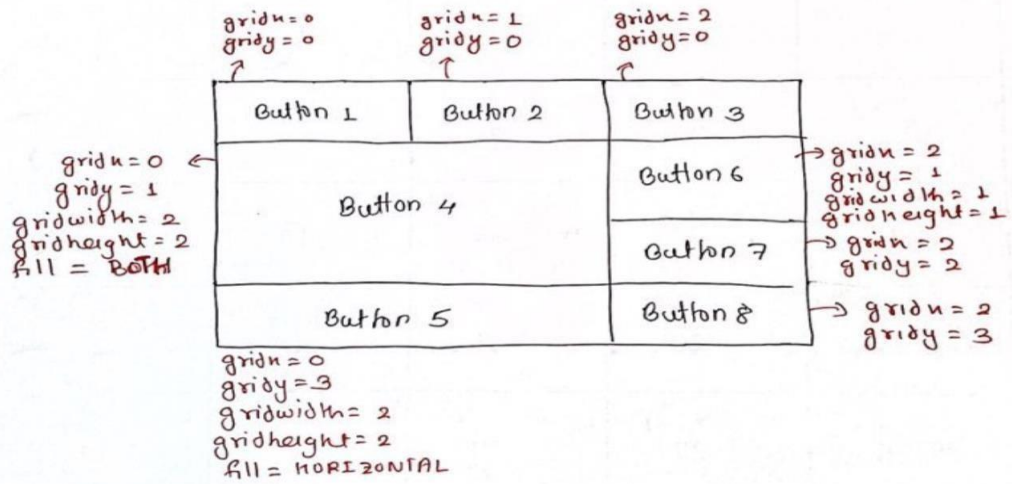
1  import javax.swing.*;
2  import java.awt.*;
3  public class GridLayoutDemo
4  {
5      JFrame f1;
6      JLabel l1,l2;
7      JButton b1,b2;
8      JTextField t1,t2;
9      JPasswordField p1,p2;
10     GridLayoutDemo()
11     {
12         f1 = new JFrame(title:" grid layout");
13         f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14         f1.setSize(width:400,height:400);
15         f1.setLayout(new GridLayout(rows:4,cols:2,hgap:5,vgap:5));
16
17         l1 = new JLabel(text:" Username");
18         l2 = new JLabel(text:" Password");
19         b1 = new JButton(text:"OK");
20         b2 = new JButton(text:"Cancel");
21         t1 = new JTextField();
22         t2 = new JTextField();
23         p1 = new JPasswordField();
24         p2 = new JPasswordField();
25
26         f1.add(l1);
27         f1.add(l2);
28         f1.add(t1);
29         f1.add(p1);
30         f1.add(t2);
31         f1.add(p2);
32         f1.add(b1);
33         f1.add(b2);
34
35
36         f1.setVisible(b:true);
37     }
38     public static void main(String args[])
39     {
40         new GridLayoutDemo();
41     }
42 }

```



GridBagLayout

- More powerful than GridLayout.
- Size of each component can be controlled.
- It has a single constructor
- It uses a helper class "GridBagConstraints" and its properties gridx, gridy, gridwidth, gridheight, fill etc.

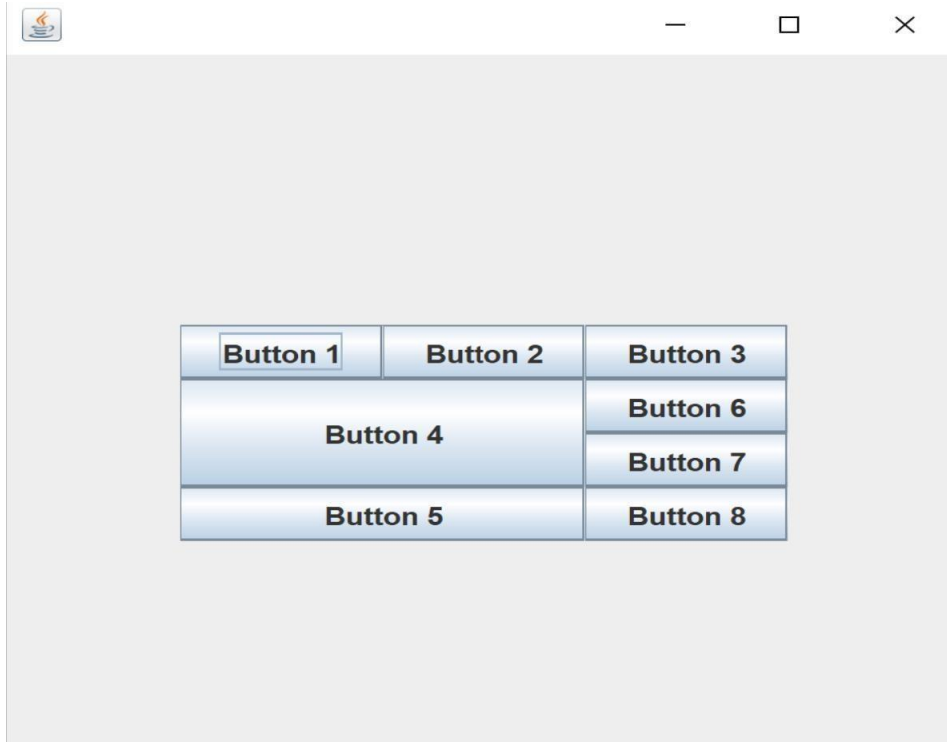


```
1  import javax.swing.*;
2  import java.awt.*;
3  public class GridBagDemo
4  {
5      JFrame f1;
6      JButton b1,b2,b3,b4,b5,b6,b7,b8;
7      GridBagDemo()
8      {
9          f1 = new JFrame();
10         f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11         f1.setSize(width:400,height:400);
12
13         GridBagLayout g1 = new GridBagLayout();
14         f1.setLayout(g1);
15         GridBagConstraints gbc = new GridBagConstraints();
16
17         b1 = new JButton(text:"Button 1");
18         gbc.gridx = 0;
19         gbc.gridy= 0;
20         g1.setConstraints(b1, gbc);
21         f1.add(b1);
22
23         b2 = new JButton(text:"Button 2");
24         gbc.gridx = 1;
25         gbc.gridy = 0;
26         g1.setConstraints(b2, gbc);
27         f1.add(b2);
28
29         b3 = new JButton(text:"Button 3");
30         gbc.gridx = 2;
31         gbc.gridy = 0;
32         g1.setConstraints(b3, gbc);
33         f1.add(b3);
34
35         b4 = new JButton(text:"Button 4");
36         gbc.gridx = 0;
37         gbc.gridy = 1;
38         gbc.gridwidth = 2;
39         gbc.gridheight = 2;
40         gbc.fill = GridBagConstraints.BOTH;
41         g1.setConstraints(b4, gbc);
42         f1.add(b4);
```

```

44         b5 = new JButton(text: "Button 5");
45         gbc.gridx = 0;
46         gbc.gridy = 3;
47         gbc.gridwidth = 2;
48         gbc.gridheight = 1;
49         gbc.fill = GridBagConstraints.BOTH;
50         g1.setConstraints(b5, gbc);
51         f1.add(b5);
52
53         b6 = new JButton(text: "Button 6");
54         gbc.gridx = 2;
55         gbc.gridy = 1;
56         gbc.gridwidth = 1;
57         gbc.gridheight = 1;
58         g1.setConstraints(b6, gbc);
59         f1.add(b6);
60
61         b7 = new JButton(text: "Button 7");
62         gbc.gridx = 2;
63         gbc.gridy = 2;
64         //gbc.gridwidth = 1;
65         //gbc.gridheight = 1;
66         g1.setConstraints(b7, gbc);
67         f1.add(b7);
68
69         b8 = new JButton(text: "Button 8");
70         gbc.gridx = 2;
71         gbc.gridy = 3;
72         //gbc.gridwidth = 1;
73         //gbc.gridheight = 1;
74         g1.setConstraints(b8, gbc);
75         f1.add(b8);
76
77         f1.setVisible(b:true);
78     }
79     Run | Debug
80     public static void main(String args[])
81     {
82         new GridBagDemo();
83     }

```



Border Layout:

A BorderLayout lays out a container, arranging its components to fit into five regions: NORTH, SOUTH, EAST, WEST and CENTER. For each region, it may contain no more than one component. When adding different components, we need to specify the orientation of it to be the one of the five region.

For BorderLayout, it can be constructed like below

- **BorderLayout():** Construct a border layout with no gaps between components.
- **BorderLayout(int hgap, int vgap):** Construct a border layout with specified gaps between components.

```

1  import javax.swing.*;
2  import java.awt.*;
3  public class BorderLayoutDemo
4  {
5      JFrame f1;
6      JButton b1,b2,b3,b4,b5;
7      BorderLayoutDemo()
8      {
9          f1 = new JFrame();
10         f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11         f1.setSize(width:400,height:400);
12         f1.setLayout(new BorderLayout(hgap:10,vgap:10));
13
14         b1 = new JButton(text:"Button 1");
15         b2 = new JButton(text:"Button 2");
16         b3 = new JButton(text:"Button 3");
17         b4 = new JButton(text:"Button 4");
18         b5 = new JButton(text:"Button 5");
19
20         f1.add(b1, BorderLayout.NORTH);
21         f1.add(b2, BorderLayout.SOUTH);
22         f1.add(b3, BorderLayout.WEST);
23         f1.add(b4, BorderLayout.CENTER);
24         f1.add(b5, BorderLayout.EAST);
25
26         f1.setVisible(b:true);
27     }
28     public static void main(String args[])
29     {
30         new BorderLayoutDemo();
31     }
32 }

```



Card Layout

The card layout manager displays the container we pass to it as cards, where each container can have different layout manager. Only one card is visible at a time and the container acts as stack of cards. We can give each card a name and can move from card to card by using `show()` method.

Commonly used constructor

`CardLayout()`: Creates a new card layout with gaps of size zero.

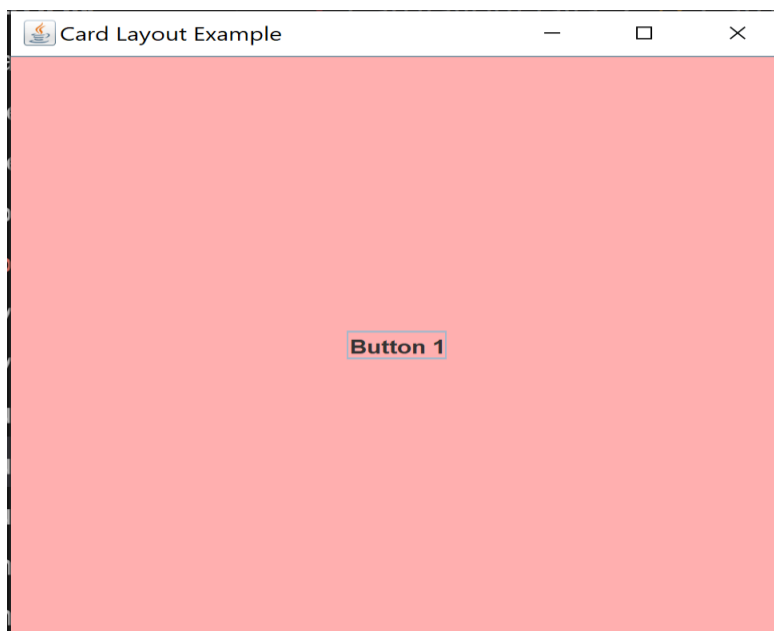
`CardLayout(int hgap, int vgap)`: Creates a new card layout with the specified horizontal and vertical gaps.

```
1  import java.awt.CardLayout;
2  import java.awt.Color;
3  import java.awt.event.ActionEvent;
4  import java.awt.event.ActionListener;
5  import javax.swing.*;
6
7  class CardLayoutDemo implements ActionListener {
8      JFrame f1;
9      JButton b1, b2, b3, b4;
10     CardLayout cl;
11
12     CardLayoutDemo() {
13         f1 = new JFrame(title:"Card Layout Example");
14         f1.setSize(width:400, height:400);
15         f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
16
17         cl = new CardLayout();
18         f1.setLayout(cl);
19
20         b1 = new JButton(text:"Button 1");
21         b2 = new JButton(text:"Button 2");
22         b3 = new JButton(text:"Button 3");
23         b4 = new JButton(text:"Button 4");
24
25         b1.setBackground(Color.pink);
26         b2.setBackground(Color.GREEN);
27         b3.setBackground(Color.blue);
28         b4.setBackground(Color.red);
29     }
30 }
```

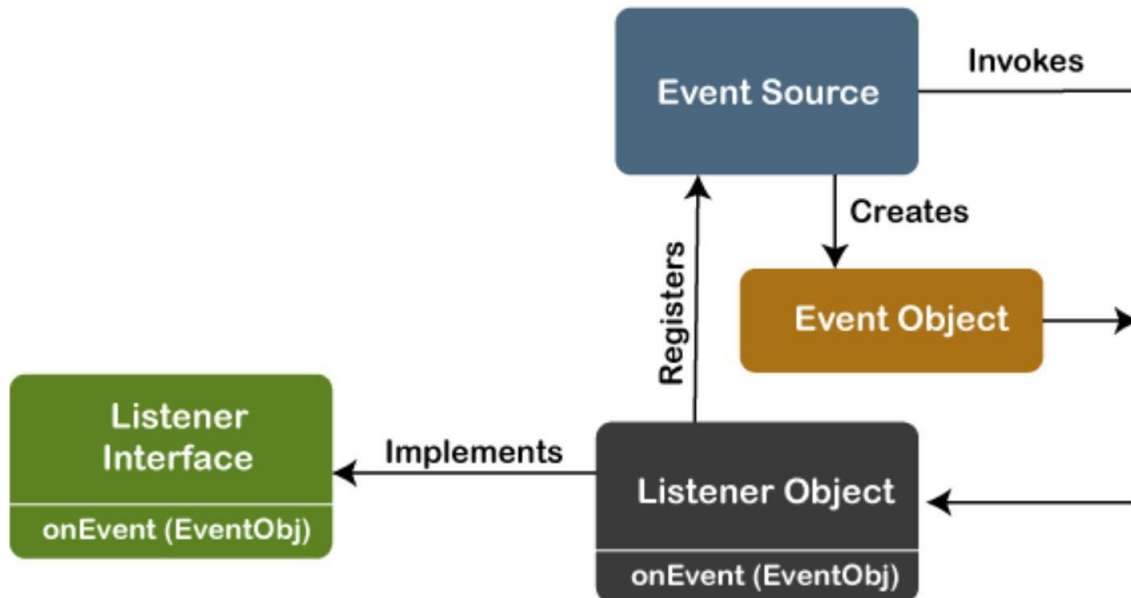
```

30      // Add components to the frame with unique identifiers
31      f1.add(b1, constraints:"1");
32      f1.add(b2, constraints:"2");
33      f1.add(b3, constraints:"3");
34      f1.add(b4, constraints:"4");
35
36      // Make the frame visible
37      f1.setVisible(b:true);
38
39      // Add action listeners to the buttons
40      b1.addActionListener(this);
41      b2.addActionListener(this);
42      b3.addActionListener(this);
43      b4.addActionListener(this);
44  }
45
46  Run | Debug
47  public static void main(String[] args) {
48      new CardlayoutDemo();
49  }
50
51  public void actionPerformed(ActionEvent e) {
52      cl.next(f1.getContentPane());
53  }

```



Event Delegation model



- The GUI programming is inherently event-driven; whenever a user initiates an activity such as a mouse activity, clicks, scrolling, etc., each is known as an event that is mapped to a code to respond to functionality to the user. This is known as event handling.
- The modern approach for event processing is based on the Delegation Model. It defines a standard and compatible mechanism to generate and process events. In this model, a source generates an event and forwards it to one or more listeners. The listener waits until it receives an event. Once it receives the event, it is processed by the listener and returns it. The UI elements are able to delegate the processing of an event to a separate function.
- The key advantage of the Delegation Event Model is that the application logic is completely separated from the interface logic.
- In this model, the listener must be connected with a source to receive the event notifications. Thus, the events will only be received by the listeners who wish to receive them. So, this approach is more convenient than the inheritance-based event model

Design goals of the event delegation model are as following

- It is easy to learn and implement.
- It supports a clean separation between application and GUI code.

- It provides robust event handling program code which is less error-prone (strong compile-time checking)
- It is Flexible, can enable different types of application models for event flow and propagation.
- It enables run-time discovery of both the component-generated events as well as observable events
- It provides support for the backward binary compatibility with the previous model

Event handling:

An event is a signal to the program that something has happened. It can be triggered by typing in a text field, selecting an item from the menu etc. The action is initiated outside the scope of the program and it is handled by a piece of code inside the program. Events may also be triggered when timer expires, hardware and software failure occurs, operation completes, counter is increased or decreased by a value etc.

- **Event handler**

The code that performs a task in response to an event is called event handler.

- **Event Handling**

It is process of responding to events that can occur at any time during execution of a program.

- **Event Source**

It is an object that generates the events. Usually the event source is a button or the other component that the user can click but any swing component can be an event source. The job of the event source is to accept registrations, get events from the user and call the listener's event handling methods.

Event Source	Description
Button	Generate action events when button is pressed.
CheckBox	Generates item events when the check box is selected or deselected
Choice	Generates item events when the choice is changed.
List	Generates action events when an item is double clicked; generates item events when an item is selected or deselected
Menu Item	Generates action events when a menu item is selected, generates item events when a checkable menu item is selected or deselected.
Scrollbar	Generates adjustment events when the scroll bar is manipulated
Text components	Generates text events when the user enters a character.
Window	Generates window events when a window is activated, closed deactivated, disconified, iconified, opened or quit.

- **Event listener**

It is an object that watch for (i.e listen for) events and handles them when they occur. It is basically a consumer that receives events from the source. To sum up, the job of an event listener is to implement the interface, register with the source and provide the event handling.

- **Listener Interface**

It is an interface which contains methods that the listener must implement and the source of the event invokes when the event occurs.

Event classes	Listener Interface
<u>ActionEvent</u> • When button is clicked. • MenuItem is clicked. • ListItem is doubled clicked. • Enter key is pressed in a text field.	<u>ActionListener</u> Public void actionPerformed(Action Event e)
<u>ItemEvent</u> • When list item, radio button, checkbox, item in a combo box is clicked.	<u>ItemListener</u> Public void itemStateChange(ItemEvent e)
<u>FocusEvent</u> • When a textfield looses or gain keyboard focus.	<u>FocusListener</u> Public void focusGained(FocusEvent e) Public void focusLost(FocusEvent e)
<u>KeyEvent</u> • When a key is pressed in a text field or text area.	<u>KeyListener</u> Public void keyPressed(keyEvent e). Public void keyReleased(keyEvent e). Public void keyTyped(keyEvent e).
<u>Container Event</u> • When a component is added to or removed from container.	<u>Container Listener</u> Public void componentAdded(Container Event e) Public void componentRemoved(ContainerEvent e)
<u>Mouse Event</u> • When the mouse is clicked, enters a component, exist a component, is pressed or is released	<u>Mouse Listener</u> Public void mouseClicked(MouseEvent e). Public void mouseEntered(MouseEvent e). Public void mouseExited(MouseEvent e). Public void mousePressed(MouseEvent e). Public void mouseReleased(MouseEvent e).

<u>Window event</u> When a window is activated, closed, deactivated, deiconified, iconified, opened or quit.	<u>Window Listener</u> Void windowActivated(WindowEvent e) Void windowClosed(WindowEvent e) Void windowClosing(WindowEvent e) Void windowDeactivated(WindowEvent e) Void windowDeiconified(WindowEvent e) Void windowIconified(WindowEvent e) Void windowOpened(WindowEvent e)

Handling Action Event

The class *ActionEvent* is defined in *java.awt.event* package. The *ActionEvent* is generated when button is clicked or the item of a list is double clicked. The object that implements the *ActionListener* interface gets this *ActionEvent* when the event occurs and hence must handle the event by overriding *actionPerformed()* method of *ActionListener* interface.

```
1  import java.awt.FlowLayout;
2  import java.awt.GridLayout;
3  import java.awt.event.ActionEvent;
4  import java.awt.event.ActionListener;
5  import javax.swing.*;
6
7  class EventHandling2 implements ActionListener
8  {
9      JFrame f1;
10     JButton b1, b2;
11     JTextField t1, t2, t3;
12     JLabel l1, l2, l3;
13
14     EventHandling2()
15     {
16         f1 = new JFrame(title:"BCA");
17         f1.setSize(width:400, height:400);
18         f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
19         f1.setLayout(new GridLayout(rows:4,cols:2));
20
21         t1 = new JTextField(columns:10);
22         t2 = new JTextField(columns:10);
23         t3 = new JTextField(columns:10);
24         t3.setEditable(b:false); // Make the result field read-only
25
26         b1 = new JButton(text:"add");
27         b2 = new JButton(text:"subtract");
28
29         l1 = new JLabel(text:"  num 1");
30         l2 = new JLabel(text:"  num 2");
31         l3 = new JLabel(text:"  Result");
32     }
```

```

33         // Add components to the frame
34         f1.add(l1);
35         f1.add(t1);
36         f1.add(l2);
37         f1.add(t2);
38
39         f1.add(l3);
40         f1.add(t3);
41         f1.add(b1);
42         f1.add(b2);
43
44         // Make the frame visible
45         f1.setVisible(true);
46
47         // Add action listeners to the buttons
48         b1.addActionListener(this);
49         b2.addActionListener(this);
50     }
51
52     Run | Debug
53     public static void main(String args[])
54     {
55         new EventHandling2();
56     }

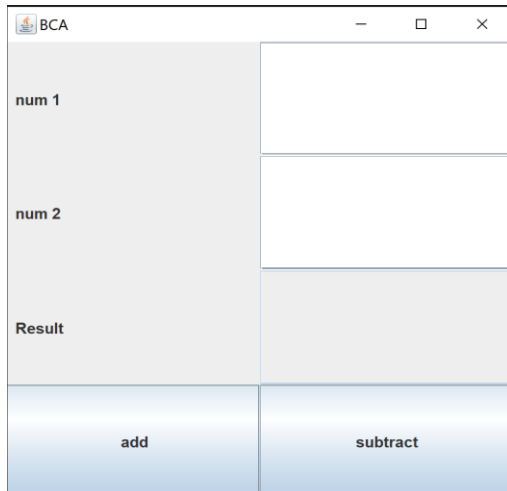
```

```

57     public void actionPerformed(ActionEvent e)
58     {
59         int a, b, c;
60
61         a = Integer.parseInt(t1.getText());
62         b = Integer.parseInt(t2.getText());
63
64         if (e.getSource() == b1) //Code to be executed when the event is triggered by the button 'b1'
65         {
66             c = a + b;
67         }
68         else if (e.getSource() == b2)
69         {
70             c = a - b;
71         }
72         else
73         {
74             c = 0;
75         }
76
77         t3.setText(String.valueOf(c));
78     }
79 }

```

Output



Key Event:

An event which indicates that a keystroke occurred in a component. This low level event is generated by a component object (such as a text field) when a key is pressed, released or typed. The event is passed to every KeyListener or KeyAdapter object which registered to receive such events using the component's addKeyListener method.

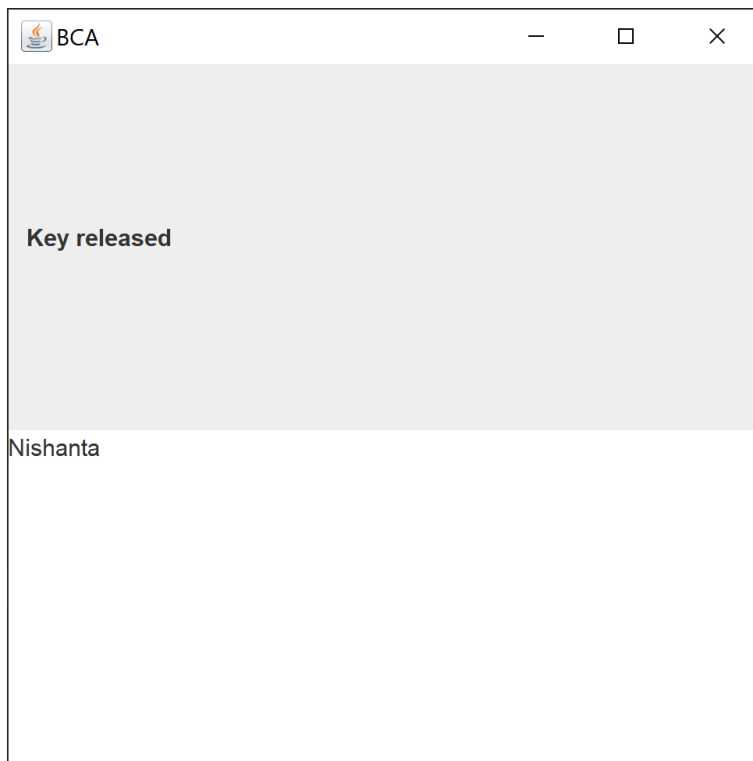
```
1  import java.awt.*;
2  import java.awt.event.KeyEvent;
3  import java.awt.event.KeyListener;
4  import javax.swing.*;
5
6  class keyevent1 extends JFrame implements KeyListener
7  {
8      JFrame f1;
9      JLabel l1;
10     JTextArea t1;
11     keyevent1()
12     {
13         f1 = new JFrame("BCA");
14
15         f1.setSize(width:400, height:400);
16         f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17         f1.setLayout(new GridLayout(2,2,hgap:10,vgap:10));
18
19         l1 = new JLabel("Email");
20         t1 = new JTextArea();
21
22         f1.add(l1);
23         f1.add(t1);
24
25         f1.setVisible(true);
26         t1.addKeyListener(this);
27     }
28 }
```

```

28
29     public void keyPressed(KeyEvent e)
30     {
31         l1.setText(text:"    Key pressed");
32     }
33     public void keyReleased(KeyEvent e)
34     {
35         l1.setText(text:"    Key released");
36     }
37     public void keyTyped(KeyEvent e)
38     {
39         l1.setText(text:"    Key typed");
40     }
41     Run | Debug
42     public static void main(String args[])
43     {
44         new keyevent1();
45     }

```

output



Mouse Event

The MouseEvent Interface represents events that occur due to the user interacting with a pointing device(such as a mouse). Common events using this interface include click, double click, mouseup, mousedown. MouseListener and MouseMotionListener is an interface in a java.awt.event package. Mouse event sare of two types. MouseListener handles the events when the mouse is not in motion.

The abstract functions are:

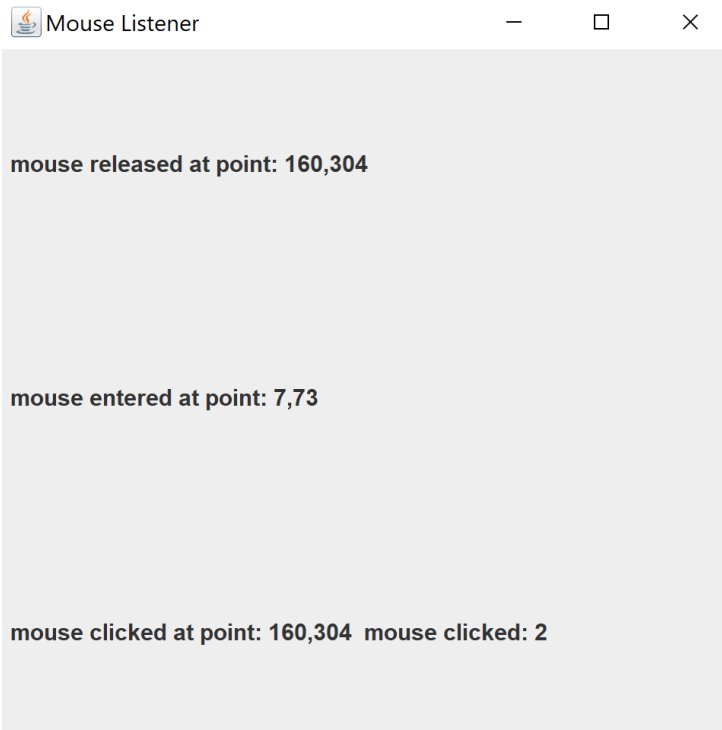
- Void mouseReleased(MouseEvent e): Mouse key is released.
- Void mouseClicked(MouseEvent e): Mouse key is pressed / released.
- Void mouseExited(MouseEvent e): Mouse exited the component.
- Void mouseEntered(MouseEvent e): Mouse entered the component.
- Void mouse pressed(MouseEvent e): Mouse key is pressed

```
1  import java.awt.*;
2  import java.awt.event.MouseEvent;
3  import java.awt.event.MouseListener;
4  import javax.swing.*;
5
6  class MouseEvent1 extends JFrame implements MouseListener
7  {
8      static JLabel l1, l2, l3;
9      public static void main(String args[])
10     {
11         JFrame f1 = new JFrame(title:"Mouse Listener");
12         f1.setSize(width:400,height:400);
13         f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14         JPanel p = new JPanel();
15         p.setLayout(new GridLayout(rows:3,cols:1,hgap:5,vgap:5));
16
17         l1 = new JLabel(text:" no event 1: ");
18         l2 = new JLabel(text:" no event 2: ");
19         l3 = new JLabel(text:" no event 3: ");
20         MouseEvent1 m = new MouseEvent1();
21
22         p.add(l1);
23         p.add(l2);
24         p.add(l3);
25         f1.add(p);
26
27         f1.addMouseListener(m);
28
29         f1.setVisible(b:true);
30     }
```



```
31 public void mousePressed(MouseEvent e)
32 {
33     l1.setText(" mouse pressed at point: " + e.getX()+" "+e.getY());
34 }
35 public void mouseReleased(MouseEvent e)
36 {
37     l1.setText(" mouse released at point: " + e.getX()+" "+e.getY());
38 }
39 public void mouseExited(MouseEvent e)
40 {
41     l2.setText(" mouse exited at point: " + e.getX()+" "+e.getY()+"\n");
42 }
43 public void mouseEntered(MouseEvent e)
44 {
45     l2.setText(" mouse entered at point: " + e.getX()+" "+e.getY()+"\n");
46 }
47 public void mouseClicked(MouseEvent e)
48 {
49     l3.setText(" mouse clicked at point: " + e.getX()+" "+e.getY()+" mouse clicked: "+e.getClickCount());
50 }
51 }
```

Output



Focus Events

The class **FocusEvent** is defined in **java.awt.event** package. Focus events occur when any component gains or loses input focus on a GUI. Focus applies to all components that can receive input. To handle a focus event, a class must implement the **FocusListener** interface or extend **FocusAdapter** class. The object that implements the **FocusListener** interface gets this **FocusEvent** when the event occurs and hence must handle the event by overriding **focusGained()** and **focusLost()** methods of **FocusListener** interface. These objects are invoked automatically for a component, which is registered with **FocusListener**, when focus is gained or lost by it respectively.

```
1  import java.awt.*;
2  import java.awt.event.FocusEvent;
3  import java.awt.event.FocusListener;
4  import javax.swing.*;
5
6  class FocusEvent1 extends JFrame implements FocusListener
7  {
8      static JTextField t1, t2, t3;
9      static JLabel l1, l2, l3;
10     static JButton b1;
11
12     public FocusEvent1()
13     {
14         JFrame f1 = new JFrame(title:"Focus Listener");
15         f1.setSize(width:400,height:400);
16         f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17         //JPanel p = new JPanel();
18         f1.setLayout(new GridLayout(rows:4,cols:2,hgap:15,vgap:15));
19
20         l1 = new JLabel(text:" First value ");
21         l2 = new JLabel(text:"  Second value ");
22         l3 = new JLabel(text:"   Third value ");
23
24         t1 = new JTextField(columns:10);
25         t2 = new JTextField(columns:10);
26         t3 = new JTextField(columns:10);
27         t3.setEditable(b:false);
```

```

29         b1 = new JButton(text:"Multiply");
30
31         f1.add(l1);
32         f1.add(t1);
33         f1.add(l2);
34         f1.add(t2);
35         f1.add(l3);
36         f1.add(t3);
37         f1.add(b1);
38         //f1.add(p);
39
40
41         t1.addFocusListener(this);
42         t2.addFocusListener(this);
43         b1.addFocusListener(this);
44
45         f1.setVisible(b:true);
46     }
47
48     Run | Debug
49     public static void main(String args[])
50     {
51         new Focusevent1();
52     }

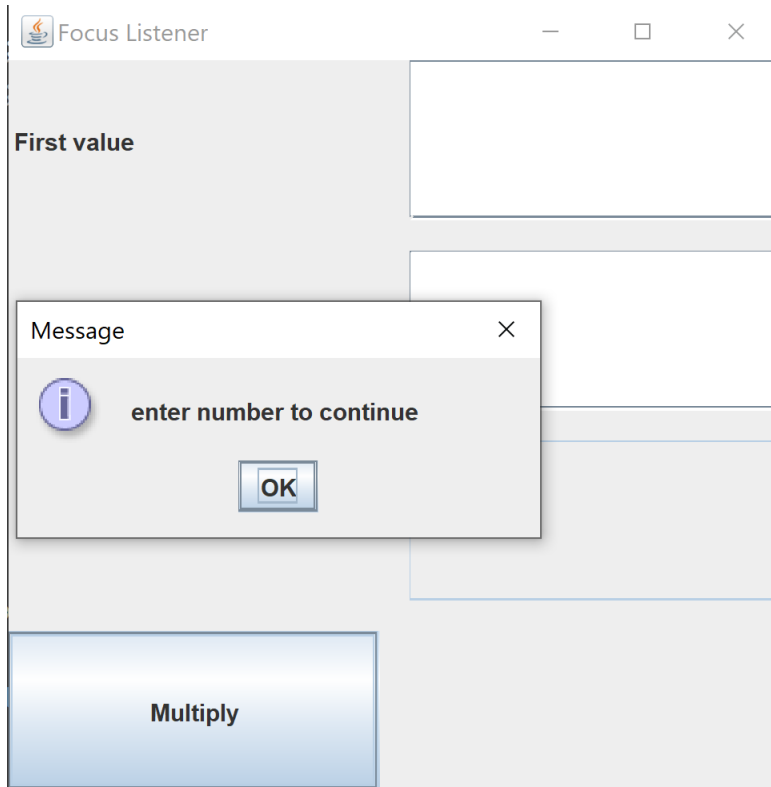
```

```

53     public void focusGained(FocusEvent e)
54     {
55         int a, b, c;
56
57         a = Integer.parseInt(t1.getText());
58         b = Integer.parseInt(t2.getText());
59
60         if (e.getSource() == b1) //Code to be executed when the event is triggered by the button 'b1'
61         {
62             c = a * b;
63             t3.setText(String.valueOf(c));
64         }
65     }
66
67
68     public void focusLost(FocusEvent e)
69     {
70         if(e.getSource() == t1 && t1.getText().equals(anObject:""))
71         {
72             JOptionPane.showMessageDialog(this, message:"enter number to continue");
73             t1.requestFocus();
74         }
75
76         if(e.getSource() == t2 && t2.getText().equals(anObject:""))
77         {
78             JOptionPane.showMessageDialog(this, message:"enter number to continue");
79             t2.requestFocus();
80         }
81     }
82 }

```

Output



ItemEvent

Item event is generated whenever user selects or deselects a selectable object such as radio button, checkbox or list. To handle an item event, a class must implement the **ItemListener** interface. There is no corresponding adapter class for this listener because there is only one method declared within it. The object that implements the **ItemListener** interface gets item event when the event occurs and hence must handle the event by overriding, **itemStateChanged()** method of **ItemListener** interface. The **stateChange** of any **ItemEvent** instance takes one of the following values:

ItemEvent SELECTED

ItemEvent DESELECTED

```

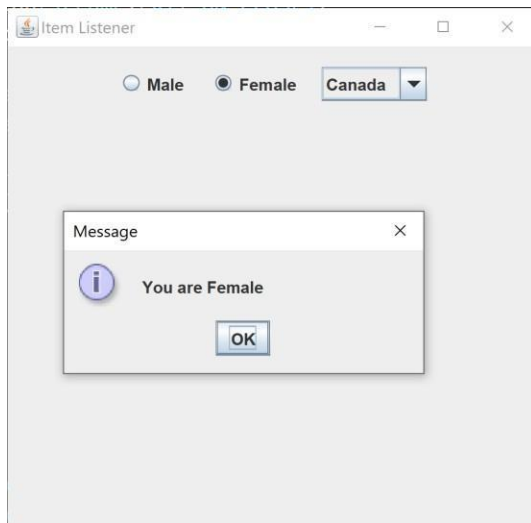
1  import java.awt.*;
2  import java.awt.event.ItemEvent;
3  import java.awt.event.ItemListener;
4  import javax.swing.*;
5
6  class Itemevent1 extends JFrame implements ItemListener
7  {
8      JFrame f1;
9      static JRadioButton rb1, rb2;
10     static JComboBox<String> cb;
11     static ButtonGroup bg;
12
13     Itemevent1()
14     {
15         f1 = new JFrame(title:"Item Listener");
16         f1.setSize(width:400, height:400);
17         f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
18         f1.setLayout(new FlowLayout(FlowLayout.CENTER, hgap:15, vgap:15));
19
20         rb1 = new JRadioButton(text:"Male");
21         rb2 = new JRadioButton(text:"Female");
22         bg = new ButtonGroup();
23         String country_name[] = {"Nepal", "India", "Canada", "Bhutan", "Thailand"};
24         cb = new JComboBox<>(country_name);
25
26         bg.add(rb1);
27         bg.add(rb2);
28
29         f1.add(rb1);
30         f1.add(rb2);
31         f1.add(cb);

```

```

33         rb1.addItemListener(this);
34         rb2.addItemListener(this);
35         cb.addItemListener(this);
36
37         f1.setVisible(b:true);
38     }
39
40     Run | Debug
41     public static void main(String[] args)
42     {
43         new Itemevent1();
44     }
45
46     public void itemStateChanged(ItemEvent e)
47     {
48         if (e.getSource() == rb1 && e.getStateChange() == ItemEvent.SELECTED)
49         {
50             JOptionPane.showMessageDialog(this, message:"You are Male");
51         }
52         else if (e.getSource() == rb2 && e.getStateChange() == ItemEvent.SELECTED)
53         {
54             JOptionPane.showMessageDialog(this, message:"You are Female");
55         }
56         else if (e.getSource() == cb && e.getStateChange() == ItemEvent.SELECTED)
57         {
58             JOptionPane.showMessageDialog(this, "Your country is " + cb.getSelectedItem());
59         }
60     }

```



Adapter class

- Java adapter classes provide the default implementation of listener interfaces.
- If we inherit adapter class, we will not be forced to provide the implementation of all the methods of listener interfaces. So it saves code.
- Event adapter classes are abstract class that provides some methods used for avoiding the heavy coding.
- Adapter class is defined for the listener that has more than one abstract methods.
- A source generates an event and sends it to one or more listeners registered with the source.
- Once event is received by the listener, they process the event and then return.
- Events are supported by a number of java packages, like java.util, java.awt and java.awt.event.

Java.awt.event Adapter classes

Adapter Class	Listener Interface
WindowAdapter	WindowListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener
MouseMotionAdapter	MouseMotionListener
FocusAdapter	FocusListener
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener

Window Event by using adapter class

The class **WindowEvent** is defined in java.awt.event package. This is generated whenever there is a change in the window state such as minimized, maximized, activate, deactivated, opened, closing, closed. Window event is generated by components such as JFrame, JInternalFrame, JDialog etc. To handle a window event, a class must implement the **WindowListener** interface or extend WindowAdapter class. The object that implements the **WindowListener** interface gets the window when the event occurs and hence must handle the event by overriding **windowClosing()**, **WindowClosed()**, **WindowIconified()**, **WindowDeiconified()**, **windowActivated()**, **WindowDeactivated()** and **WindowOpened()** methods of **WindowListener** interface.


```

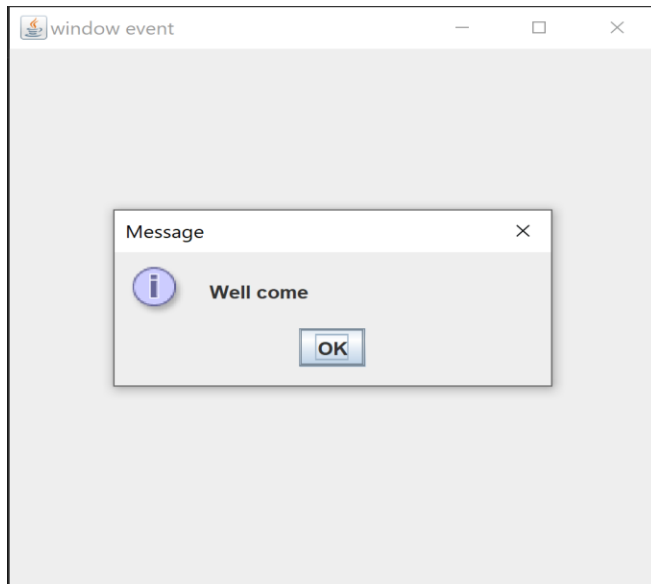
1  import java.awt.*;
2  import javax.swing.*;
3  import java.awt.event.*;
4
5  public class Adapter extends WindowAdapter
6  {
7      JFrame f1;
8      Adapter()
9      {
10         f1 = new JFrame(title:"window event");
11         f1.setSize(width:400, height:400);
12         f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
13
14         f1.addWindowListener(this);
15         f1.setVisible(b:true);
16     }
17
18     public void windowClosing(WindowEvent e)
19     {
20         JOptionPane.showMessageDialog(f1, message:"Good Bye");
21     }
22
23     public void windowOpened(WindowEvent e)
24     {
25         JOptionPane.showMessageDialog(f1, message:"Well come");
26     }
27
28     public void windowIconified(WindowEvent e)
29     {
30         JOptionPane.showMessageDialog(f1, message:"See you later");
31     }

```

```

33     public void windowDeiconified(WindowEvent e)
34     {
35         JOptionPane.showMessageDialog(f1, message:"Well come back");
36     }
37
38     Run | Debug
39     public static void main(String[] args)
40     {
41         new Adapter();
42     }

```

Introducing Graphics

The awt includes several methods that support graphics. All graphics are drawn relative to a window. This can be the main window of an application or a child window. The origin of each window is at the top – left corner and is 0.0. Coordinates are specified in pixels. All output to a widow takes place through a graphics context.

A graphics context is encapsulated by the graphics class. Here are two ways in which a graphics context can be obtained:

- It is passed to a method, such as `paint()` or `update()` as an argument.
- It is returned by the `getGraphics()` method of component.

Drawing Lines

Lines are drawn by means of the `drawLine()` method.

Void `drawLine(int startX, int startY, int endX, endY)`

Drawing Rectangle

The **drawRect()** and **fillRect()** methods display an outlined and filled rectangle, respectively they are shown as:

Void drawRect(int left, int top, int width, int height)

Void fillRect(int left, int top, int width, int height)

Here the upper – left corner of the rectangle is at left, top. To draw a rounded rectangle, use **drawRoundRect()** and **fillRoundRect()** are shown below:

Void drawRoundRect(int left, int top, int width, int height, int xDiam, int yDiam)

Void fillRoundRect(int left, int top, int width, int height, int xDiam, int yDiam)

Drawing Ellipse and Circle

To draw an ellipse, use **drawOval()**. To fill an ellipse, use **fillOval()**. These methods are shown here:

Void drawOval(int left, int top, int width, int height)

Void fillOval(int left, int top, int width, int height)

Drawing Arcs

Arcs can be drawn with **drawArc()** and **fillArc()**, shown here

Void drawArc(int left, int top, int width, int height, int startAngle, int sweepAngle)

Void fillArc(int left, int top, int width, int height, int startAngle, int sweepAngle)

Drawing polygon

It is possible to draw arbitrarily shaped figures using **drawPolygon()** and **fillPolygon()** shown here:

Void drawPolygon(int x[], int y[], int numPoints)

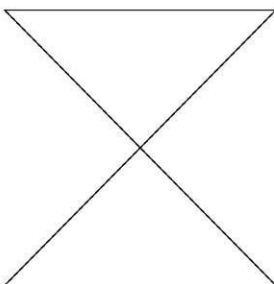
Void fillPolygon(int x[], int y[], int numPoints)

```
1  import java.awt.*;
2  import javax.swing.*;
3
4  public class Drawing extends JFrame
5  {
6      public Drawing()
7      {
8          setSize(width:400, height:700);
9          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
10         setVisible(b:true);
11     }
12
13     public void paint(Graphics g)
14     {
15         // Draw lines
16         g.drawLine(x1:80, y1:40, x2:100, y2:90);
17         g.drawLine(x1:20, y1:90, x2:100, y2:40);
18         g.drawLine(x1:40, y1:45, x2:250, y2:80);
19
20         // Draw rectangles
21         g.drawRect(x:20, y:150, width:60, height:50);
22         g.fillRect(x:110, y:150, width:60, height:50);
23         g.drawRoundRect(x:200, y:150, width:60, height:50, arcWidth:15, arcHeight:15);
24         g.fillRoundRect(x:290, y:150, width:60, height:50, arcWidth:30, arcHeight:40);
25
26         //Draw ellipse and circle
27         g.drawOval(x:20, y:250, width:50, height:50);
28         g.fillOval(x:100, y:250, width:75, height:50);
29         g.drawOval(x:200, y:260, width:100, height:40);
```

```

31 //Draw arcs
32 g.drawArc(x:20, y:350, width:70, height:70, startAngle:0, arcAngle:180);
33 g.fillArc(x:70, y:350, width:70, height:70, startAngle:0, arcAngle:75);
34
35 //Draw polygon
36 int xpoints[] = {20,200,20,200,20};
37 int ypoints[] = {450,450,650,650,450};
38 int num = 5;
39
40 g.drawPolygon(xpoints, ypoints,num);
41
42 }
43
44 Run | Debug
45 public static void main(String[] args) {
46     new Drawing();
47 }

```



JavaFX

- JavaFX is a java library used to develop desktop application as well as Rich Internet Application (RIA).
- The application built in javaFX, can run on multiple platforms including web, mobile and desktops.
- JavaFX is intended to replace swing in java applications as a GUI framework. It provides more functionalities than swing.
- Like swing, javaFX also provides own components and does not depend upon the operating system.
- It is lightweight and hardware accelerated. It supports various operating system including windows, Linux and MacOS.

JavaFX Vs Swing

Swing	javaFX
Swing is the standard toolkit for java developer in creating GUI.	JavaFX provides platform support for creating desktop application.
Swing has a more sophisticated set of GUI components.	JavaFX has a decent number of UI components available but lesser than what Swing provides.
Swing is a legacy library that fully features and provides pluggable UI components	JavaFX has UI components that are still evolving with a more advanced look and feel.
Swing has a UI component library and act as a legacy.	javaFX has several components built over Swing.
Swing does not have support for customization using CSS and XML.	JavaFX has a support for customization using CSS and XML.
With swing, it is very difficult to create beautiful 3 – D application.	With javaFX one can also create beautiful 3 – D application.

Simple JavaFx Program

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.scene.Scene;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;

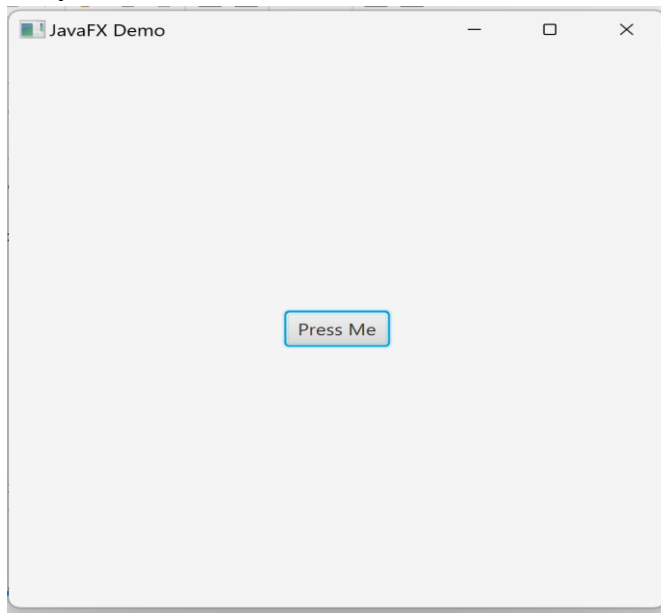
public class JavaFX_Application extends Application
{
    public static void main(String[] args)
    {
        launch(args);
    }

    @Override
    public void start(Stage primaryStage) throws Exception
    {
        Button bt1 = new Button("Press Me");
        StackPane root = new StackPane();
        root.getChildren().add(bt1);

        Scene S1 = new Scene(root, 400, 400);
        primaryStage.setScene(S1);
        primaryStage.setTitle("JavaFX Demo");
        primaryStage.show();

        bt1.setOnAction(new EventHandler<ActionEvent>()
        {
            public void handle(ActionEvent e)
            {
                System.out.println("Hello Everest");
            }
        });
    }
}
```

Output



JavaFx Layout

- Layout panes are containers which are used for flexible and dynamic arrangements of UI controls within a scene graph of a JavaFX application.
- As a window is resize, the layout pane automatically repositions and resize the nodes it contains.
- JavaFx has the following built-in layout panes: FlowPane, HBox, VBox, BorderPane, GridPane, StackPane, TilePane, AnchorPane.

FlowPane

- FlowPane position nodes in a row or a column, where the nodes are wrapped when they all cannot be shown.
- The default orientation of a flow pane is Horizontal.
- The class name FlowPane of the package javafx.scene.layout represents the FlowPane layout.
- There are 8 constructors in the class that are given below:
 - FlowPane()
 - FlowPane(Double Hgap, Double Vgap)
 - FlowPane(Double Hgap, Double Vgap, Node.....Children)
 - FlowPane(Node.....Children)
 - FlowPane(Orientation orientation)
 - FlowPane(Orientation orientation, Double Hgap, Double Vgap)
 - FlowPane(Orientation orientation, Double Hgap, Double Vgap, Node.....Children)
 - FlowPane(Orientation orientation, Node.....Children)

Example

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.FlowPane;
import javafx.scene.Scene;
import javafx.geometry.Orientation;

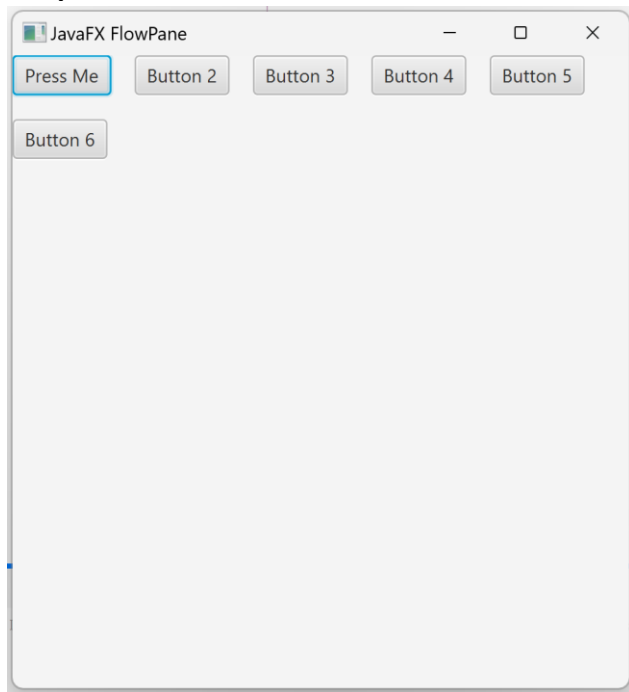
public class Flowpane extends Application
{
    public static void main(String[] args)
    {
        launch(args);
    }

    @Override
    public void start(Stage primaryStage) throws Exception
    {
        Button bt1 = new Button("Press Me");
        Button bt2 = new Button("Button 2");
        Button bt3 = new Button("Button 3");
        Button bt4 = new Button("Button 4");
        Button bt5 = new Button("Button 5");
        Button bt6 = new Button("Button 6");

        FlowPane root = new FlowPane(Orientation.HORIZONTAL, 15 , 15);
        root.getChildren().add(bt1);
        root.getChildren().add(bt2);
        root.getChildren().add(bt3);
        root.getChildren().add(bt4);
        root.getChildren().add(bt5);
        root.getChildren().add(bt6);

        Scene S1 = new Scene(root, 400, 400);
        primaryStage.setScene(S1);
        primaryStage.setTitle("JavaFX FlowPane");
        primaryStage.show();
    }
}
```


Output



HBox

- If we use HBox in the layout in our application, all the nodes are set in a single horizontal row.
- The class named HBox of the package `javafx.scene.layout` represents the HBox pane.
- The HBox class contains two constructors that are given below
 - `HBox()`: creates HBox layout with 0 spacing
 - `HBox(Double spacing)`: Create HBox layout with a spacing value

Example

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.HBox;
import javafx.scene.Scene;
import javafx.geometry.Insets;
import javafx.geometry.Pos;

public class HBox1 extends Application
{
    public static void main(String[] args)
    {
        launch(args);
    }

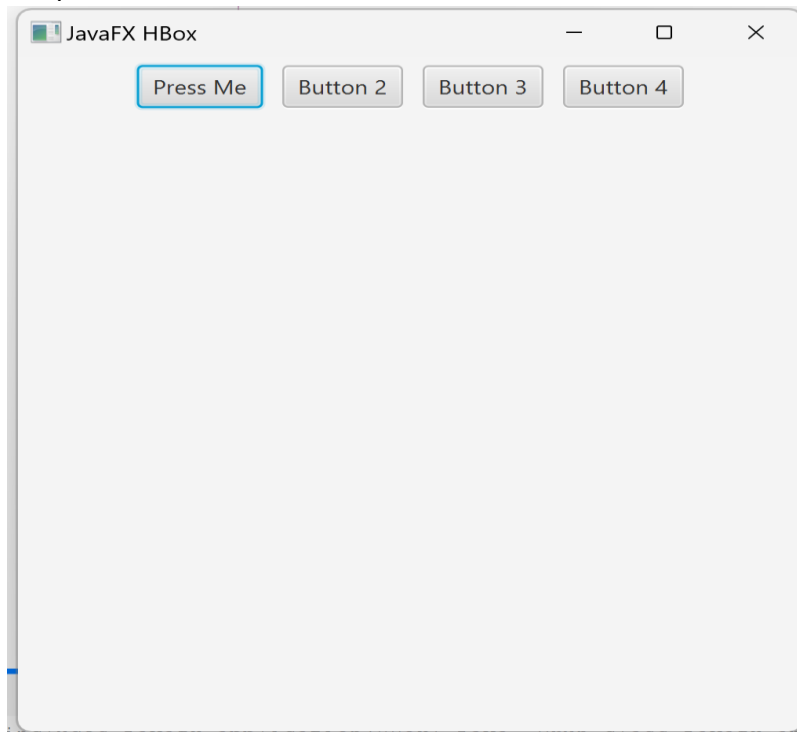
    @Override
    public void start(Stage primaryStage) throws Exception
    {
        Button bt1 = new Button("Press Me");
        Button bt2 = new Button("Button 2");
        Button bt3 = new Button("Button 3");
        Button bt4 = new Button("Button 4");

        HBox root = new HBox(10);
        root.setPadding(new Insets(5));
        root.setAlignment(Pos.BASELINE_CENTER);

        root.getChildren().add(bt1);
        root.getChildren().add(bt2);
        root.getChildren().add(bt3);
        root.getChildren().add(bt4);

        Scene S1 = new Scene(root, 400, 400);
        primaryStage.setScene(S1);
        primaryStage.setTitle("JavaFX HBox");
        primaryStage.show();
    }
}
```

Output



VBox

- Instead of arranging the nodes in horizontal row, VBox layout pane arranges the nodes in a single vertical column.
- It is represented by `javafx.scene.layout.VBox` class.
- The VBox class contains four constructors that are given below:
 - `VBox()`: Creates layout with 0 spacing
 - `VBox(Double Spacing)`: creates layout with a spacing value of double type
 - `VBox(Double spacing, Node Children)`: creates a layout with specified spacing among the specified child nodes.
 - `VBox(Node Children)`: creates a layout with the specified nodes having 0 spacing among them.

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.VBox;
import javafx.scene.Scene;
import javafx.geometry.Insets;
import javafx.geometry.Pos;

public class VBox1 extends Application
{
    public static void main(String[] args)
    {
        launch(args);
    }

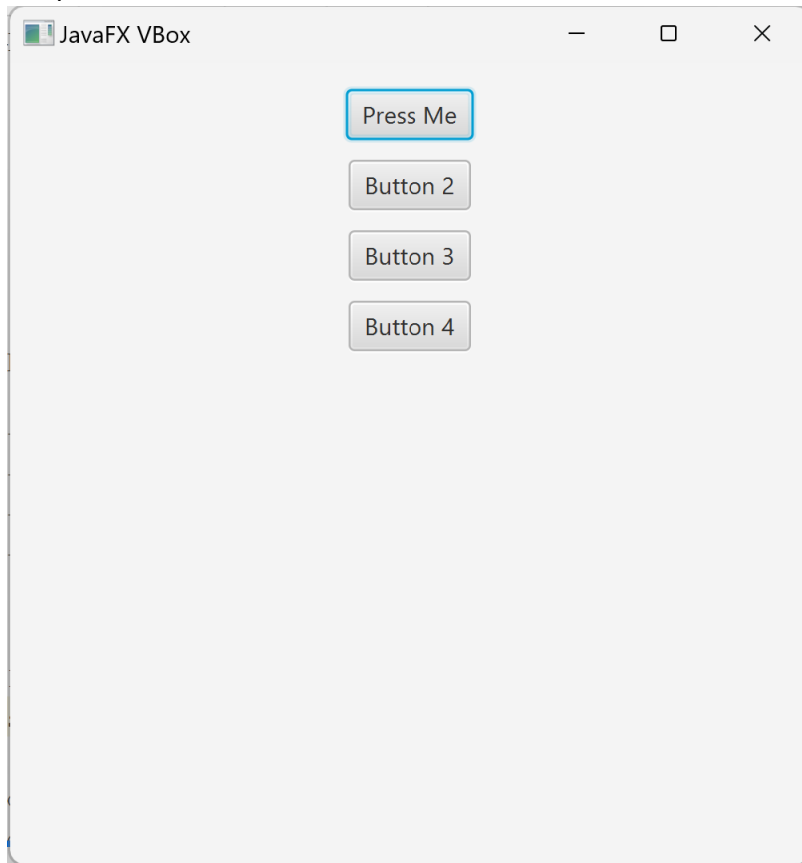
    @Override
    public void start(Stage primaryStage) throws Exception
    {
        Button bt1 = new Button("Press Me");
        Button bt2 = new Button("Button 2");
        Button bt3 = new Button("Button 3");
        Button bt4 = new Button("Button 4");

        VBox root = new VBox(10);
        root.setPadding(new Insets(5));
        root.setAlignment(Pos.BASELINE_RIGHT);

        root.getChildren().add(bt1);
        root.getChildren().add(bt2);
        root.getChildren().add(bt3);
        root.getChildren().add(bt4);

        Scene S1 = new Scene(root, 400, 400);
        primaryStage.setScene(S1);
        primaryStage.setTitle("JavaFX VBox");
        primaryStage.show();
    }
}
```

Output



BorderPane

- BorderPane layout children in top, left, right, bottom and center positions.
- It is represented by `javafx.scene.layout.BorderPane` class.
- This class provides various methods like `setRight()`, `setLeft()`, `setTop()`, `setBottom()`, `setCenter()` which are used to set the position for the specified nodes.
- We need to instantiate `BorderPane` class to create the `BorderPane` layout.
- There are the following constructors in the class:
 - `BorderPane()`: create the empty layout
 - `BorderPane(Node Center)`: create the layout with the center node
 - `BorderPane(Node Center, Node Top, Node Right, Node Bottom, Node Left)`: create the layout with all the nodes.

Example

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.BorderPane;
import javafx.scene.Scene;
import javafx.geometry.Insets;

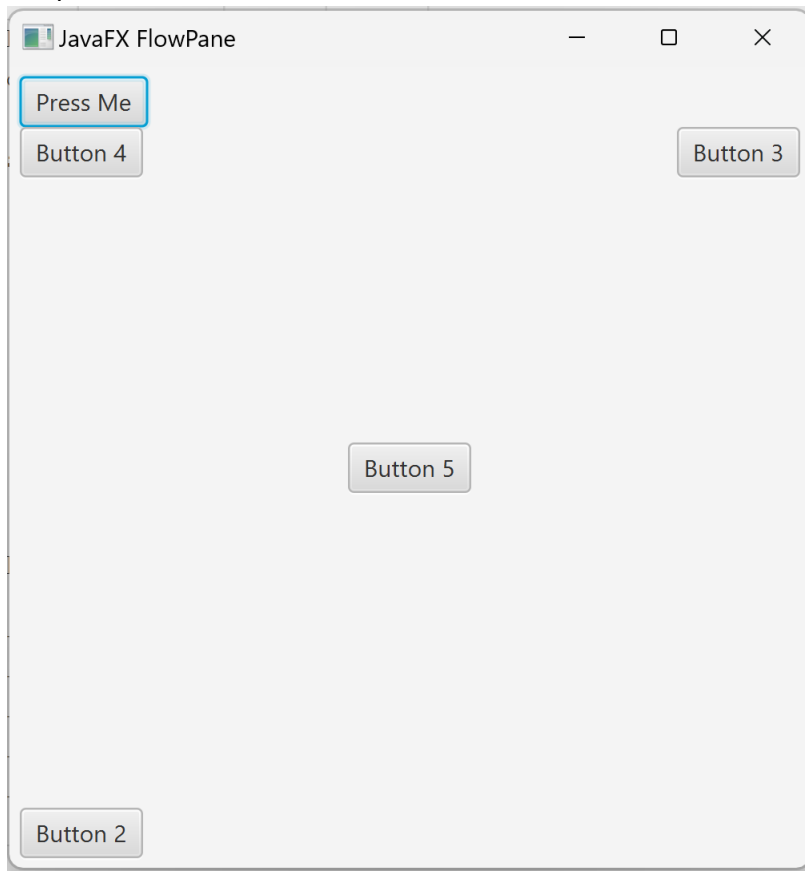
public class BorderPanel1 extends Application
{
    public static void main(String[] args)
    {
        launch(args);
    }

    @Override
    public void start(Stage primaryStage) throws Exception
    {
        Button bt1 = new Button("Press Me");
        Button bt2 = new Button("Button 2");
        Button bt3 = new Button("Button 3");
        Button bt4 = new Button("Button 4");
        Button bt5 = new Button("Button 5");

        BorderPane root = new BorderPane();
        root.setPadding(new Insets(5));
        root.setTop(bt1);
        root.setBottom(bt2);
        root.setRight(bt3);
        root.setLeft(bt4);
        root.setCenter(bt5);

        Scene S1 = new Scene(root, 400, 400);
        primaryStage.setScene(S1);
        primaryStage.setTitle("JavaFX FlowPane");
        primaryStage.show();
    }
}
```

Output



GridPane

- GridPane layout allow us to add the multiple nodes in a multiple rows and columns.
- It is seen as a flexible grid of rows and columns where nodes can be placed in any cell of the grid.
- It is represented by `javafx.scene.layout.GridPane` class.
- We just need to instantiate this class to implement GridPane.
- The class contains only one constructor that is given below
 - `GridPane()`: creates a gridpane with 0 Hgap and Vgap

Example

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.layout.GridPane;
import javafx.scene.Scene;
import javafx.geometry.Insets;

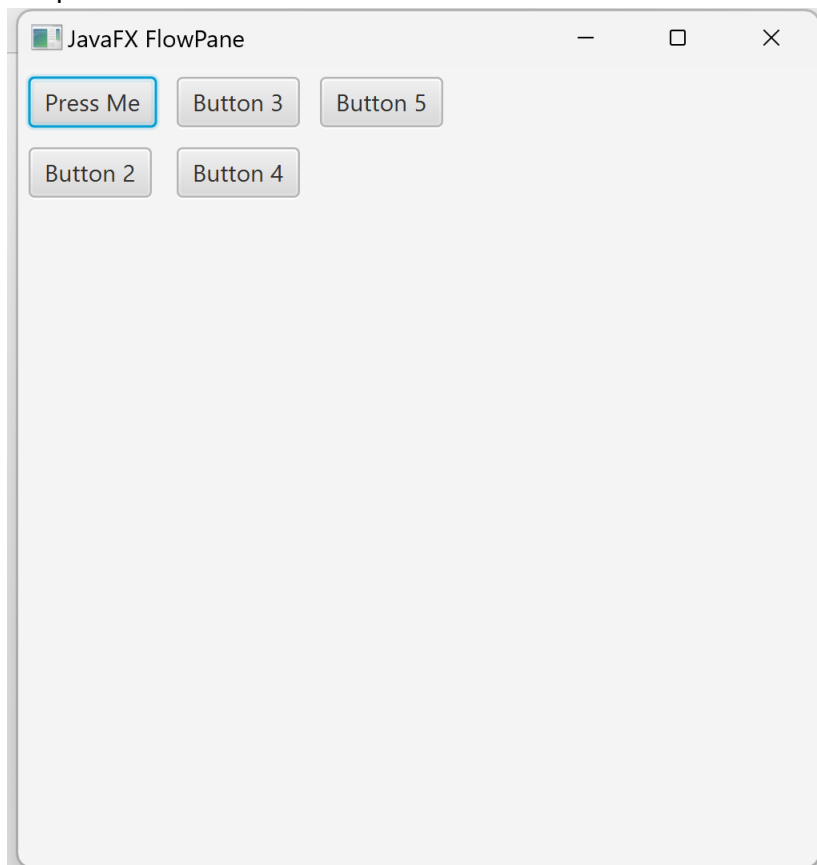
public class GridPanel extends Application
{
    public static void main(String[] args)
    {
        launch(args);
    }

    @Override
    public void start(Stage primaryStage) throws Exception
    {
        Button bt1 = new Button("Press Me");
        Button bt2 = new Button("Button 2");
        Button bt3 = new Button("Button 3");
        Button bt4 = new Button("Button 4");
        Button bt5 = new Button("Button 5");

        GridPane root = new GridPane();
        root.setPadding(new Insets(5));
        root.setVgap(10);
        root.setHgap(10);
        root.add(bt1, 0, 0);
        root.add(bt2, 0, 1);
        root.add(bt3, 1, 0);
        root.add(bt4, 1, 1);
        root.add(bt5, 2, 0);

        Scene S1 = new Scene(root, 400, 400);
        primaryStage.setScene(S1);
        primaryStage.setTitle("JavaFX FlowPane");
        primaryStage.show();
    }
}
```


Output



JavaFx UI Controls

- The graphical user interface of every desktop application mainly considers UI elements, layout and behavior.
- The UI elements are the one which are actually shown to the user for interaction or information exchange.
- Layout defines the organization of the UI elements on the screen
- However, the package `javafx.scene.control` provides all the necessary classes for the UI components like Button, Label etc.
- Every class represents a specific UI controls and defines some methods for their styling.
- Basic javaFX UI controls are discussed below.

1. Label

- Label is a non – editable text control.
- `Javafx.scene.control.label` class represents label control.
- As the name suggests, the label is the component that is used to place any text information on the screen.
- It is mainly used to describe the purpose of the other components to the user. You cannot set a focus on the label using the Tab key.
- Constructors of label class are listed below.
 - **Label():** creates an empty label
 - **Label(String text):** creates label with the supplied text

- **Label(String text, Node graphics):** create label with the supplied text and graphics.

2. TextField

- A TextField object is a text component that allow for the editing of a single line of text. Text Field is basically used to get the input form the user in the form of text.
- `Javafx.scene.control.TextField` represent the textfield.
- Constructors of the TextField class are listed below:
 - **TextField():** Creates a new TextField with empty text content.
 - **TextField(String s):** creates a new TextField with a initial Text.

3. PasswordField

- A javaFX PasswordField control enables users of a javaFX application to enter password which can then be read by the application
- The PasswordField control does not shoe the texted entered to it.
- Instead it shows a dot for each character entered.
- The javaFX PasswordField control is represented by the class `Javafx.scene.control.PasswordField`.
- Constructors of the PasswordField class are listed below
 - **PasswordField()**

4. Button

- A javaFX button control enable a javaFX application to have some action executed when the application user clicks the button.
- The javaFX button control is represented by the class `Javafx.scene.control.Button`.
- A javaFX button can have a text and an icon on it which indicate to the user what clicking the button will do.
- Constructor for creating buttons are listed below:
 - **Button():** Creates a button with an empty string for its label.
 - **Button(String text):** creates a button with the specified text as its label.
 - **Button(String text, Node graphic):** Creates a button with the specified text and icon for its label.

Example

```
import static javafx.application.Application.launch;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.TextField;
import javafx.scene.control.PasswordField;
import javafx.scene.control.Label;
import javafx.scene.layout.GridPane;
import javafx.scene.Scene;
import javafx.geometry.Insets;

public class LoginForm extends Application
{
    public static void main(String[] args)
    {
        launch(args);
    }
    @Override
    public void start(Stage primaryStage) throws Exception
    {
        TextField t1 = new TextField();
        t1.setStyle("-fx-font: normal bold 20px 'serif'");
        PasswordField p1 = new PasswordField();
        p1.setStyle("-fx-font: normal bold 20px 'serif'");

        Label l1 = new Label("Email");
        Label l2 = new Label("Password");

        Button bt1 = new Button("Submit");
        bt1.setStyle("-fx-background-color: blue; -fx-text-fill: white;");
        Button bt2 = new Button("Clear");
        bt2.setStyle("-fx-background-color: blue; -fx-text-fill: white;");
```

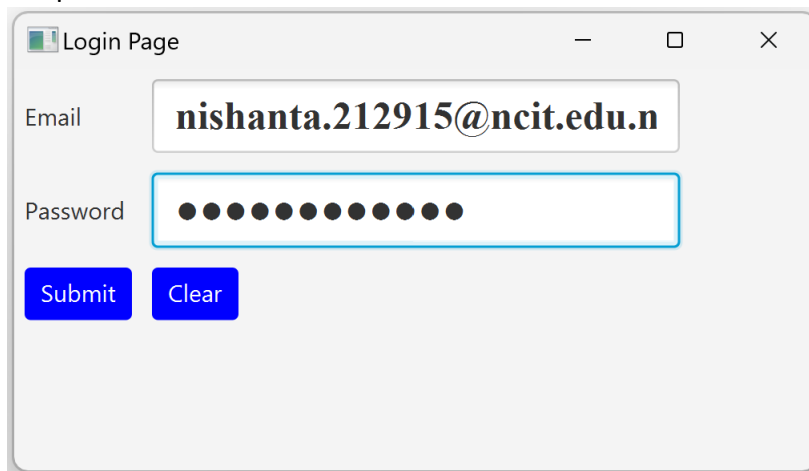
```

GridPane root = new GridPane();
root.setPadding(new Insets(5));
root.setVgap(10);
root.setHgap(10);
root.add(l1, 0, 0);
root.add(t1, 1, 0);
root.add(l2, 0, 1);
root.add(p1, 1, 1);
root.add(bt1, 0, 2);
root.add(bt2, 1, 2);

Scene S1 = new Scene(root, 400, 200);
primaryStage.setScene(S1);
primaryStage.setTitle("Login Page");
primaryStage.show();
}
}

```

Output



5. RadioButton

- The Radio Button is used to provide various options to the user. The user can only choose one option among all.
- A radio button is either selected or deselected. It can be used in a scenario of multiple choice question on the quiz where only one option needs to be chosen by the student.
- Class `javafx.scene.control.RadioButton` represents the javaFX RadioButton.
- Constructors for creating a RadioButton is given below
 - **`RadioButton()`**: Creates a radio button with an empty string for its label.
 - **`RadioButton(String Text)`**: Creates a radio button with the specified text as its label.

6. CheckBox

- The checkbox is used to provide more than one choices to the user.
- It can be used in a scenario where the user prompted to select more than one option or the user wants to select multiple options.
- We have to instantiate `javafx.scene.control.CheckBox` class to implement
- Constructors for creating a check box is given below
 - **`checkBox()`**: creates a checkbox with an empty string for its label.
 - **`checkbox(String text)`**: Creates a check box with specified text as its label.

7. Hyperlink

- In javaFx, we can use hyper-link to refer the web pages.
- It is similar to anchor link in HTML. `Javafx.scene.control.HyperLink` class provides all the necessary methods to deal with javaFX hyper-links.
- We can configures a Hyperlink to perform some action when the user click it.
- Constructors used to create hyperlink is given below
 - **`HyperLink()`**: creates a hyperlink with no label.
 - **`HyperLink(String text)`**: creates a hyperlink with the specified text as it label.
 - **`HyperLink(String text, Node graphic)`**: create a hyperlink with the specified text and graphic as its label.

8. ListView

- The `ListView` is a graphical user interface component used for displaying a list of items from which a user can select desired items.
- In javaFX, the list view is represented by a class named `ListView` which is a part of `javafx.scene.control` package.
- We have option to select its orientation, allowing it to be displayed either vertically or horizontally.
- We can also able to set the preferred height and width of list as our need. To do this

```
setPrefHeight(double height)
setPrefWidth(double width)
```

or we can use a single call to set both dimension as

```
setPrefSize(double width, double height)
```
- Constructors used to create a `ListView` class is given below:
 - **`ListView()`**: It is the default constructor that construct a vertical list view.
 - **`ListView(ObservableList<type> listItems)`**: It construct a new vertical list view with specified list items.

9. ChoiceBox

- ChoiceBox is a part of the javafx package.
- ChoiceBox shows a set of items and allows the user to select a single choice and it will show the currently selected item on the top.
- ChoiceBox by default has no selected item unless otherwise selected.
- Constructor used to create a ChoiceBox is given below
 - **ChoiceBox()**: creates a new empty ChoiceBox.
 - **ChoiceBox(ObservableList<type> items)**: creates a new ChoiceBox with the given set of items.

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.stage.Stage;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.scene.control.Button;
import javafx.scene.control.TextField;
import javafx.scene.control.DatePicker;
import javafx.scene.control.RadioButton;
import javafx.scene.control.ListView;
import javafx.scene.control.CheckBox;
import javafx.scene.control.ChoiceBox;
import javafx.scene.control.ToggleGroup;
import javafx.scene.control.Label;
import javafx.scene.layout.GridPane;
import javafx.scene.Scene;
import javafx.geometry.Insets;

public class Registration_Form extends Application
{
    public static void main(String[] args)
    {
        launch(args);
    }
    @Override
    public void start(Stage primaryStage) throws Exception
    {
        Label l1 = new Label("Name");
        Label l2 = new Label("Date of Birth");
        Label l3 = new Label("Gender");
        Label l4 = new Label("Your Skill");
        Label l5 = new Label("Academic Qualification");
        Label l6 = new Label("Location");
```

```
TextField t1 = new TextField();
DatePicker dp = new DatePicker();

RadioButton rb1 = new RadioButton("Male");
RadioButton rb2 = new RadioButton("Female");
RadioButton rb3 = new RadioButton("Other");

ToggleGroup tg = new ToggleGroup();
rb1.setToggleGroup(tg);
rb2.setToggleGroup(tg);
rb3.setToggleGroup(tg);

CheckBox cb1 = new CheckBox("Java");
CheckBox cb2 = new CheckBox("Python");
CheckBox cb3 = new CheckBox("DotNet");

ObservableList<String> data = FXCollections.observableArrayList
("BSc.CSIT", "BIT", "BCA", "BE.Computer", "BE.IT", "BIM");
ListView<String> lv = new ListView (data);
lv.setPrefSize(50,100);

ChoiceBox cob = new ChoiceBox();
cob.getItems().addAll("Kathmandu", "Bhaktpur", "Pokhara", "Lalitpur", "Dhading");

Button bt1 = new Button("Submit");
Button bt2 = new Button("Clear");

GridPane root = new GridPane();
root.setPadding(new Insets(10));
root.setVgap(10);
root.setHgap(10);
```

```
root.add(l1, 0, 0);
root.add(t1, 1, 0);

root.add(l2, 0 , 1);
root.add(dp, 1, 1);

root.add(l3, 0, 2);
root.add(rb1, 1,2);
root.add(rb2, 2,2);
root.add(rb3, 3,2);

root.add(l4, 0,3);
root.add(cb1, 1,3);
root.add(cb2, 2,3);
root.add(cb3, 3,3);

root.add(l5, 0,4);
root.add(lv, 1,4);

root.add(l6, 0,5);
root.add(cob, 1,5);

root.add(bt1, 1,6);
root.add(bt2, 2,6);

Scene S1 = new Scene(root, 500, 300);
primaryStage.setScene(S1);
primaryStage.setTitle("Registration Form");
primaryStage.show();
}
```


Output:

Registration Form

Name

Date of Birth

Gender ☐ Male ☐ Female ☐ Other

Your Skill ☐ Java ☐ Python ☐ DotNet

Academic Qualification

Location

10. Menu

- JavaFX provides a Menu class to implement menus. Menu is the main component of a any application.
- In javaFX, javafx.scene.control.Menu class provides all the methods to deal with menus.
- This class need to be instantiated to create a Menu. Menu contains several menu items that are displayed when the user clicks a menu.
- The user can select a menu item after which the menu goes into a hidden state.
- Menubar is usually placed at the top of the screen which contains several menus.
- Constructor used to create a Menubar are as follows
 - MenuBar(): creates a new empty menubar.
 - MenuBar(Menu m): creates a new menubar with the given set of menu.
- Constructor of the Menu class are
 - Menu(): creates an empty menu.
 - Menu(String s): creates a menu with a string as its label.
 - Menu(String s, Node n): Construct a Menu and sets the display text with the specified text and sets the graphic Node to the given node.
 - Menu(String s, Node n, MenuItem i): Construct a menu and sets the display text with the specified text, the graphic Node to the given node and insert the given items into the item list.
- Constructor of the MenuItem class are:
 - MenuItem(): create a menuitem with no display text.
 - MenuItem(string text): Construct a menuitem and sets the display text with the specified text.
 - MenuItem(String text, Node graphic): Construct a menuitem and sets the display text with the specified text and sets the graphic Node to the given node.

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.stage.Stage;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.*;

public class JavaFX_Menu extends Application
{
    public static void main(String[] args)
    {
        launch(args);
    }

    @Override
    public void start(Stage primaryStage) throws Exception
    {
        MenuBar mb = new MenuBar();

        Menu x1 = new Menu("Veg");
        Menu x2 = new Menu("Non-veg");

        MenuItem m1 = new MenuItem("Pakoda");
        MenuItem m2 = new MenuItem("Veg-Momo");
        MenuItem m3 = new MenuItem("Veg-Chowmein");
        MenuItem s1 = new MenuItem("Chicken Korma");
        MenuItem s2 = new MenuItem("Mutton Biryani");

        x1.getItems().addAll(m1, m2, m3);
        x2.getItems().addAll(s1, s2);

        mb.getMenus().addAll(x1, x2);

        VBox root = new VBox(mb);
        Scene S1 = new Scene(root, 400, 200);
        primaryStage.setScene(S1);
        primaryStage.setTitle("Menu Example");
        primaryStage.show();
    }
}
```

Output:

